



tu technische universität dortmund

Practical Lab CoRobots

Hands-on experience with collaborative robots in real and simulated environments.

ROBOTICS | AI | PRACTICE | A BETTER TOMORROW

*Learn
Build
Experiment
Collaborate*

ROS 2

*Real Robots.
Real Skills.
Real Applications.*

ROBOT PROGRAMMING | SIMULATION (GAZEBO / RVIZ) | VISION-BASED MANIPULATION | ROS 2 INTEGRATION | HUMAN-ROBOT COLLABORATION

PRACTICE TODAY FOR A SMARTER TOMORROW

1. Simulate → 2. Program → 3. Perceive → 4. Manipulate → 5. Real-World Results

CoRobot Lab SoSe 2026

Version 25.09.2026

CoRobot Lab

apl. Prof. Dr. rer. nat Frank Hoffmann

Univ.-Prof. Dr.-Ing. Prof. h.c. Dr. h.c. Torsten Bertram
Institute of Control Theory and Systems Engineering

General Information CoRobot Lab

Curricula : MA A&R, MA SES, MA ET

Time & Place

Labs: IRF Mobile Robot Lab Area

Mo-Fr : 9:00-17:00

Start: 28.09.2026

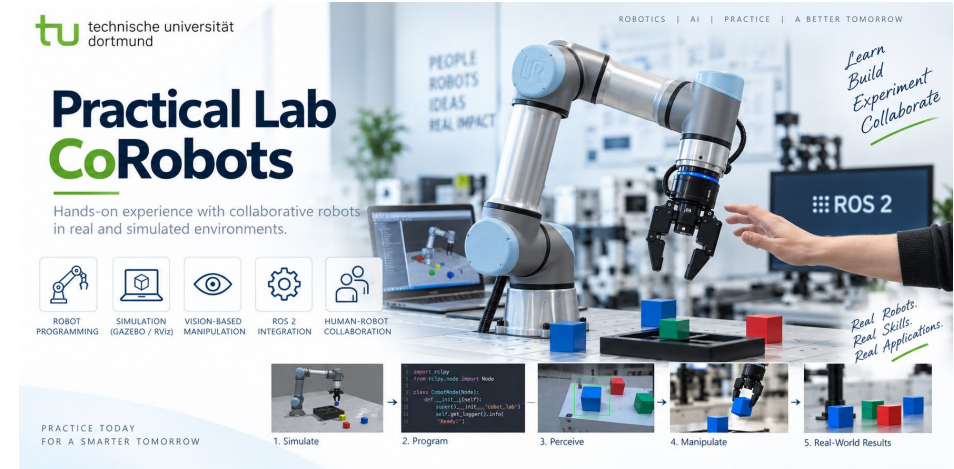


Moodle Workspace : <https://moodle.tu-dortmund.de/course/view.php?id=59336>

Teaching assistants: M.Sc. Shreyas Desikan, Kavish Punitbhai Gajjar

Requirements

- Programming in Python
- Groups of 2-3 students
- Completion and upload of solution code
- Final interview for grading



Use of LLMs for Coding

Use of LLMs for Coding in the Assignments

You are welcome to use LLMs (e.g., ChatGPT, Gemini, Copilot) to support your work – but you are responsible for understanding, verifying and analyzing the code.

1. Get Help from LLMs

You may use LLMs to ...

How can I control the myCobot 320 with ROS 2 to pick up a cube using a suction pump? Please provide Python code.



```
# Example code
import rclpy
...
# control suction pump
# move to pose
# pick and place
...
```

- ✓ Get explanations and examples
- ✓ Generate and edit code
- ✓ Ask for debugging help
- ✓ Learn concepts and interfaces

2. Verify and Understand

You are responsible for ...

- ✓ **Review** the code and check that it is correct and safe to use.
- ✓ **Understand** how the code works (e.g., ROS 2 nodes, topics, services, APIs).
- ✓ **Adapt** and **test** the code for your task.
- ✓ **Verify** the results (e.g., check robot behavior, sensor data, and outputs).
- ✓ **Document** your solution and your analysis.



3. Final Interview (Determines the Grade)

You should be able to ...



- ✓ **Explain** how you solved the assignment.
- ✓ **Describe** and **justify** important parts of your code.
- ✓ **Analyze** the robot behavior based on experiments and results (e.g., trajectories, sensor data, success rate).
- ✓ **Answer** questions about the underlying concepts (e.g., ROS 2, control, perception, planning).

⚠ Important Notes

- You **remain responsible** for the code you submit.
- Generated code must be **verified** and tested.
- You should **understand all parts** of your solution.
- Copying code without understanding is not sufficient.
- Be aware of potential errors or unsafe behavior in generated code.

👍 Good Practice

- Use LLMs as a learning and productivity tool.
- Read and understand the generated code.
- Compare different solutions and test them.
- Adapt the code to your specific task.
- Analyze and discuss the results.
- Prepare to explain your decisions and observations in the final interview.

🚫 Not Allowed

- Submitting code without understanding.
- Making up explanations of generated code.
- Using LLMs during the final interview.
- Plagiarism from other students or external sources.

🎯 Goal

Use LLMs to support your learning and development, and demonstrate in the final interview that you can

- ✓ **solve the task,**
- ✓ **understand your code, and**
- ✓ **analyze the robot behavior** based on your results.

IRF Lab Policy

Lab Safety Regulations – Overview

Robotics Research Institute (IRF)



1. Access & General Conduct



Access hours (Mon–Fri)

7:00–18:00 (winter term)
7:00–19:00 (summer term)
All visitors and students must leave by 20:00. Working later requires approval from your chair.



Access rights

- Staff receive a group key for offices and shared areas.
- Students may use computer and seminar rooms, corridors and toilets (ground floor).
- Access to other rooms, especially the lab area, requires permission of a staff member.



Unauthorized access prohibited

The lab area is prohibited for unauthorized persons.

2. Lab Usage



No unsupervised experiments
Do not conduct experiments or operate machinery in the labs without the presence of an additional qualified staff member.



Close windows

Opened windows must be closed by the person who opened them when leaving – even if others are present.



Turn off lights

The last person present must turn off the lighting (unless others are in the room).



Be considerate

Avoid unnecessary disturbances to others (noise, dust, heat, etc.).

3. Occupational Safety & Health (OSH)



Follow regulations

The public accident prevention regulations apply.
<http://www.unfallkasse-nrw.de/service/medien/regeln-und-schriften/unfallverhuetungsvorschriften.html>



Use personal protective equipment

(work shoes, gloves, goggles, etc.).



Keep corridors clear

Corridors and traffic lanes must be kept clear at all times.



Safety instruction

Undergo a safety instruction by the responsible instructor before using technical equipment, machinery or conducting hardware experiments.



Be aware of risks

Rotating or moving parts (robots, motors, propellers, etc.) can cause serious injuries. Exercise extra caution – complete shielding is not always possible.

4. Computers & Network



Authorized use only

Computers may only be used by employees and students of the RST chair (exceptions: publicly accessible computer and seminar room). Obtain username and password from Mr. Kersting.



Do not change system settings

Do not modify computer system settings yourself.



No software installation

Students are not allowed to install software on the computers by themselves.



Use for scientific purposes only

Download of software, multimedia files or data is only permitted for scientific or official purposes. Comply with license terms.



Log off

Log off if you do not use the computer for an extended period of time.

5. Technical Equipment & Facilities



Mandatory briefing

Before operating any experiment or equipment, undergo a briefing by the responsible staff according to the operating instructions.



Check before use

Perform a visual inspection and confirm the validity of the electrical equipment test label.



No unauthorized modifications

Unauthorized modifications are strictly prohibited.



Report defects

Report any damage or defect immediately to Mr. Cicek or Mr. Kersting and stop operation of faulty devices.



Keep it tidy

Exercise orderliness and cleanliness, handle equipment with care.



Turn off equipment

Turn off machinery and equipment if you leave the room, even for short periods. Never leave running experiments unattended.

6. Fire Protection



Follow fire safety regulations

The fire safety regulations of the TU Dortmund apply.



In case of fire

Refer to the signs "behavior in case of fire" displayed in the corridors.



Fire extinguishers

Fire extinguishers are available at designated locations in the IRF.



No Inflammable materials

Do not store inflammable materials in corridors or hallways.

7. Accidents & Emergencies



Render first aid

Provide first aid, but do not put yourself at risk.



Emergency call

In case of an emergency: Dial **112** and inform the institute staff.



Report accidents

All accidents must be reported to the institute management.



Company doctor

Seek medical attention from the company doctor immediately.

IRF Alarm Plan

Institut für Roboterforsch
Gebäude IRF
Otto-Hahn-Str. 8
44227 Dortmund

Alarm plan

Stand: April 2020

Fire Accident Fire brigade and rescue service 0-112 Who reports? Where has something happened? What happened? How many people are in danger or injured? Wait for further questions!						
Immediate Measures Endangered persons. Get Help. Distribute tasks.						
Other Message Director Prof. Torsten Bertram: ☎ 2760 Control room of TU-Dortmund: ☎ 3333						
First Aid <table><tr><td><i>S. Kersting</i></td><td><i>R. 121 ☎ 2119</i></td></tr><tr><td><i>S. Rückheim</i></td><td><i>R. 121 ☎ 6356</i></td></tr><tr><td><i>H. Cicek</i></td><td><i>R. 122 ☎ 4649</i></td></tr></table> First Aid EG corridor and mailroom 1.OG hallway (see marking) 2.OG hallway (see marking) 	<i>S. Kersting</i>	<i>R. 121 ☎ 2119</i>	<i>S. Rückheim</i>	<i>R. 121 ☎ 6356</i>	<i>H. Cicek</i>	<i>R. 122 ☎ 4649</i>
<i>S. Kersting</i>	<i>R. 121 ☎ 2119</i>					
<i>S. Rückheim</i>	<i>R. 121 ☎ 6356</i>					
<i>H. Cicek</i>	<i>R. 122 ☎ 4649</i>					
Fire Fighting As far as possible without danger! Fire extinguishers: laboratory, hallways and basements (see marking) Fire brigade: Sascha Kersting, Halit Cicek 						
Further Measures Keep fire doors closed. (Do not close the smoke) of the doors and windows in the danger area. Take all persons from the area / building with you. Remove the building when the alarm signal (siren or horn) is activated. Check the parking area at the parking lot (marking MPI) directly. The IRF building may only be re-entered after approval by the fire protection officer or representative. 						
Prof. T. Bertram , Prof. U. Schwiigelshohn	TU-Dortmund, Brandschutzordnung Teil C					

Learning Objectives CoRobot Lab

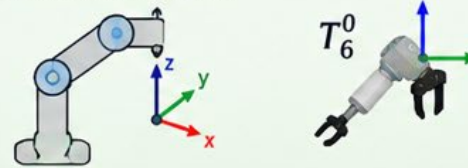
1 Robot System and Programming

- Get familiar with the Elephant myCobot 280 (collaborative robot) and its setup
- Understand the robot hardware, end-effector and workspace
- Use ROS2 for robot control and communication
- Work with the provided simulation (RViz / Gazebo) and the real robot



2 Forward and Inverse Kinematics

- Understand the robot kinematic model and coordinate frames
- Compute forward kinematics: joint angles $\rightarrow$ end-effector pose
- Solve inverse kinematics: desired pose $\rightarrow$ joint angles
- Use the robot model (URDF) in ROS2 and in MATLAB/Python



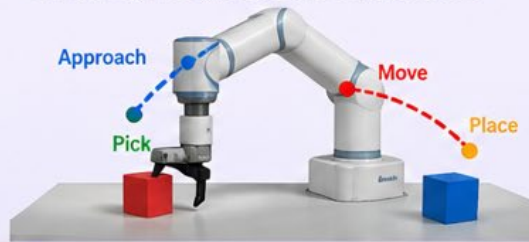
3 ROS2 Integration

- Use ROS2 for robot control and sensor integration
- Work with nodes, topics, services and actions
- Implement joint position control and Cartesian motion
- Use visualization tools (RViz) and simulation (Gazebo)
- Transfer the developed code to the real robot



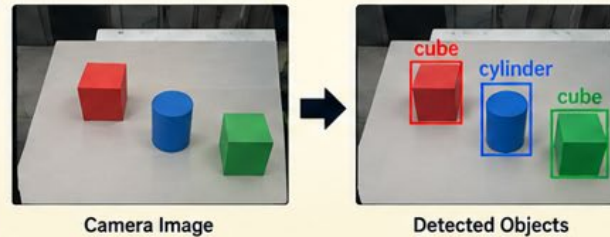
4 Trajectory Planning and Execution

- Generate and execute joint-space and Cartesian trajectories
- Plan smooth, collision-free motions
- Use motion planning tools (e.g. MoveIt 2)
- Implement pick and place trajectories with appropriate approach and retreat motions



5 Vision-based Object Detection

- Acquire and process images from a camera
- Detect and localize objects using color and shape (e.g. cubes, cylinders)
- Estimate the 3D pick position (image $\rightarrow$ world coordinates)
- Handle different objects and backgrounds robustly



6 Vision-based Pick and Place

- Combine perception, planning and control
- Detect an object, compute a grasp pose and plan a pick and place trajectory
- Execute the task on the real robot
- Evaluate the system performance and discuss limitations



Overall Learning Outcome

- Apply kinematics, ROS2, motion planning and computer vision to implement a complete pick and place application with the Elephant myCobot.
- Gain practical experience with a modern collaborative robot in simulation and on real hardware.
- Understand the integration of perception, planning and control in a robotic application.

Industrial Robot Arms vs. CoRobots

Industrial Robot Arms vs. Collaborative Robots (CoRobots)

Key differences, characteristics, and typical applications

Industrial Robot Arm

High performance for automated production



- Designed for**
High-speed, high-precision automation in industrial environments
- Focus**
Performance, payload, repeatability
- Operation**
Normally separated from humans (safety fences)
- Typical use**
High-volume, repetitive tasks in manufacturing

Collaborative Robot (CoRobot)

Work together with humans – safe and flexible



- Designed for**
Safe interaction and collaboration with humans
- Focus**
Safety, ease of use, flexibility
- Operation**
Built-in safety functions (force/torque sensing, speed and force limits)
- Typical use**
Shared workspaces, flexible production, research, education

Aspect	Industrial Robot Arm	Collaborative Robot (CoRobot)
 Payload	High (typically 10–1,000 kg)	Low to medium (typically 0.5–30 kg)
 Reach	Large (up to several meters)	Smaller (typically 0.5–1.5 m)
 Speed	High (fast cycle times)	Lower (speed limited for safety)
 Precision & Repeatability	Very high (± 0.01 – 0.1 mm)	High (± 0.1 – 1 mm)
 Safety Concept	Safety fences, light curtains, separate workspace	Integrated safety functions (force/torque sensing, power and speed limits, collision detection)
 Programming	Often requires expert knowledge (e.g., teach pendant, offline programming)	Easy and intuitive (hand-guiding, graphical interfaces, drag & drop)
 Flexibility	Optimized for fixed, repetitive tasks	Flexible, quick reconfiguration for different tasks
 Cost	Higher investment and integration cost	Lower cost and simpler integration
 Typical Environment	Industrial production lines (e.g., automotive, electronics)	Shared workspaces, small and medium enterprises, research and education

Typical Applications

Industrial Robot Arms



- Welding and assembly
- Material handling and palletizing
- Machine tending
- High-volume production
- Automotive and metal industry

CoRobots



- Pick-and-place
- Assembly of small parts
- Inspection and testing
- Research and education
- Flexible, low-volume production
- Human-robot interaction and assistance

Advantages and Limitations

Industrial Robot Arms



- High payload, speed, and precision
- Optimized for repetitive tasks
- Mature and reliable technology



- Requires safety fencing
- Less flexible
- Higher cost and integration effort
- More complex programming

CoRobots



- Safe human-robot collaboration
- Easy to program and deploy
- Flexible for different tasks
- Lower cost



- Lower payload and speed
- Less precise than industrial robots
- Not suitable for very heavy or high-speed applications

Key Takeaways



Industrial robots are optimized for high performance in automated production environments.



CoRobots are designed for safe and flexible collaboration with humans.



Both have their strengths and are complementary technologies.



CoRobots lower the entry barrier for robotics and enable new applications in research, education, and small-scale production.



The choice depends on the application requirements (payload, speed, precision, safety, flexibility, and cost).

Programming of CoRobots

Programming CoRobots – Handheld Device and Teaching

Intuitive ways to program and guide collaborative robots

1. Programming Options

CoRobots can be programmed in different ways, ranging from intuitive teaching with a handheld device to programming via graphical interfaces or code.



Handheld device (teach pendant)
Intuitive teaching and configuration



Graphical interface
Drag & drop, visual programming



Programmatic control
Python / ROS 2 for advanced tasks



Hybrid approach
Combine teaching, GUI and code

2. Handheld Device (Teach Pendant)



Typical functions

- Power on/off, enable robot
- Move joints (jog mode)
- Cartesian motion (move tool)
- Set waypoints by teaching
- Define and edit sequences
- Configure I/O and gripper
- Adjust speed and limits
- Play, stop, and step through programs
- Safeguard functions (e.g., reduced speed)

3. Teaching by Demonstration

- 1 Move to desired pose – Manually guide the robot to the desired position (teach mode).
- 2 Record waypoint – Save the current pose as a waypoint using the handheld device.
- 3 Define and execute – Create a sequence of waypoints and execute the program.



Waypoint Teaching

No.	Name	Joints
1	Home	...
2	Approach	...
3	Pick	...
4	Place	...

+ Add - Delete Edit



4. Programming Example (Python / ROS 2)

Control the CoRobot programmatically using Python and ROS 2.

```
# Example: move to a joint configuration and control gripper
from interbotix_xs_modules.arm import InterbotixManipulatorXS
from interbotix_xs_modules.gripper import InterbotixGripperXS

# connect to robot
robot = InterbotixManipulatorXS("mycobot280", "arm", "gripper")

# move to joint positions (in radians)
q = [0.0, -0.8, 1.0, 0.2, 0.0, 0.0]
robot.arm.set_joint_positions(q, moving_time=2.0)

# operate gripper
robot.gripper.close()
# or robot.gripper.open()
```

Alternative interfaces

- ROS 2 action client (NavigateToPose style)
- MoveIt 2 for motion planning
- Graphical programming environments (e.g., MoveIt Studio)

5. Execution and Control

Run, monitor, and adjust robot programs.



Execute program
Run the recorded or scripted sequence on the robot.



Adjust parameters
Set speed, acceleration and blending.



Pause and modify
Interrupt execution and edit waypoints if needed.



Monitor robot state
View joint positions, tool pose, and I/O status.

6. Typical Applications

CoRobot programming with teaching and handheld device is used for:



Pick-and-place
Simple and flexible programming of manipulation tasks



Assembly
Teaching precise waypoints for assembly operations



Inspection and testing
Programming inspection paths and sensor-based tasks



Education and research
Intuitive programming for experiments and prototyping

myCobot280pi Specs

Elephant Robotics myCobot 280 Pi

Compact 6-DOF Collaborative Robot



Lightweight and compact

Ideal for education, research and prototyping



Safe for human-robot collaboration

with integrated sensors and smooth design



Open and flexible

Supports ROS2, Python, C++, MoveIt 2



Ready to use

Includes vacuum suction pump and complete control hardware

Key Specifications

Degrees of freedom	6
Payload (recommended)	250 g
Reach	280 mm
Repeatability	± 0.5 mm
Weight	≈ 1.1 kg
Power supply	24 V DC, 5 A
Typical power consumption	≈ 60 W
Joint range	$\pm 170^\circ$ (J1-J6)
Maximum joint speed	$\approx 180^\circ/\text{s}$
Control interface	USB / Serial / Wi-Fi
Supported platforms	ROS2, Python, C++, MoveIt 2
End effector	Vacuum suction pump (included)
Mounting options	Table mount (M4), base holes, optional brackets

Dimensions and Workspace

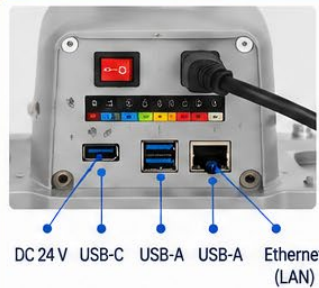


Joint Information

Joint	Range	Max. speed
J1 (Base)	$\pm 170^\circ$	$180^\circ/\text{s}$
J2 (Shoulder)	$\pm 170^\circ$	$180^\circ/\text{s}$
J3 (Elbow)	$\pm 170^\circ$	$180^\circ/\text{s}$
J4 (Wrist 1)	$\pm 170^\circ$	$180^\circ/\text{s}$
J5 (Wrist 2)	$\pm 170^\circ$	$180^\circ/\text{s}$
J6 (Wrist 3)	$\pm 170^\circ$	$180^\circ/\text{s}$



Interfaces and Connectivity



- 24 V DC power supply
- USB-C and USB-A ports
- Ethernet (LAN)
- Wi-Fi (optional)
- ROS2 support (Ubuntu 22.04/24.04)
- Compatible with MoveIt 2, RViz and Gazebo

End Effector: Vacuum Suction Pump

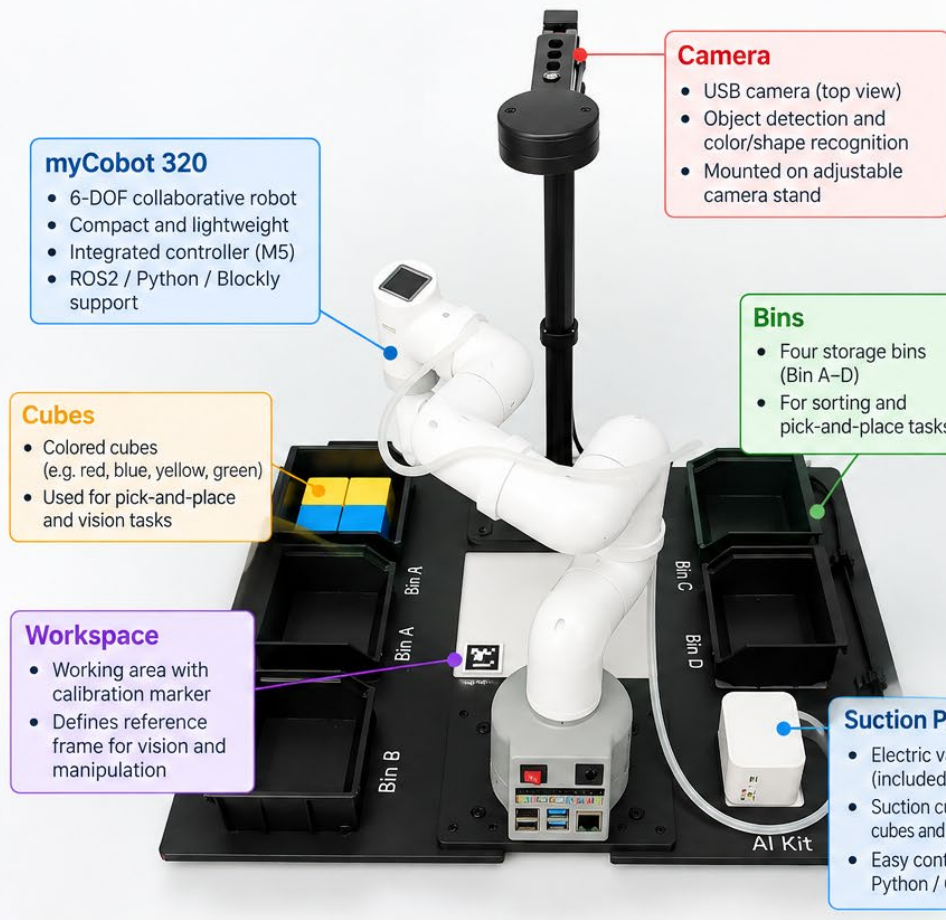


- Electric vacuum suction pump (included)
- Lightweight and compact design
- Vacuum cup for grasping objects (e.g. cubes, cylinders, small parts)
- Easy control via ROS2, Python or C++

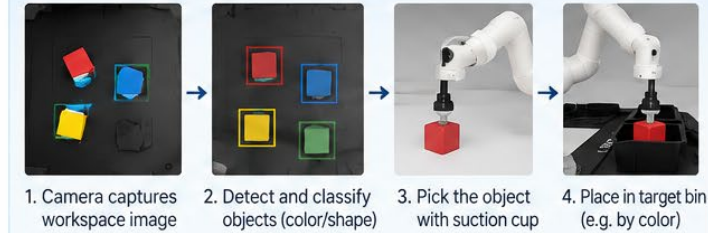
myCobot 280pi AI KIT

Elephant Robotics AI Kit for myCobot 320

Vision-based Pick-and-Place Training Platform



Use Case: Vision-Based Pick and Place



Kit Components



Learning Objectives



Ubuntu ROS Setup



Ubuntu 24.04 + ROS 2 Jazzy – System Setup

Dual-boot with Windows, installation and basic configuration for the myCobot labs

ROS 2
Jazzy



1 Prepare free space in Windows

Shrink the Windows partition (C:) to create unallocated space for Ubuntu.

Volume	Layout	Type	File System	Status
~(Disk 0 partition 1)	Simple	Basic	EFI	Healthy (EFI S...
~Windows (C:)	Simple	Basic	NTFS	Healthy (Boot...
~(Disk 0 partition 3)	Simple	Basic		Healthy (Recov...

~Disk 0				
Basic	EFI	Windows (C:)		Unallocated
931.50 GB	260 MB	650 GB NTFS		100 GB

- Open Disk Management (Win + X).
- Shrink the C: partition (recommended ≥ 80–150 GB).
- After shrinking, you should see unallocated space (e.g. 100 GB) on the same disk.

2 Create Ubuntu USB (UEFI)

Create a bootable USB stick with Ubuntu 24.04 LTS.

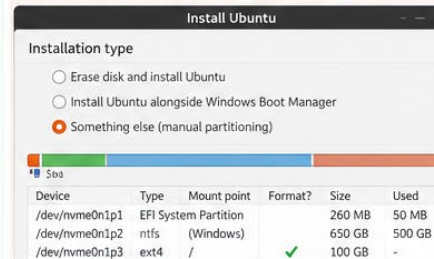


- Download Ubuntu 24.04 LTS ISO (from ubuntu.com).
- Use Rufus (Windows) to create the USB stick:

Partition scheme: GPT
Target system: UEFI (non-CSM)
File system: FAT32
- Write the ISO to the USB stick and safely eject it.

3 Boot from USB and Install Ubuntu

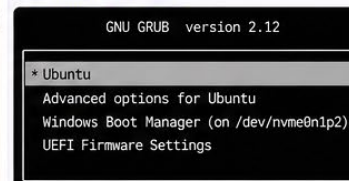
Use the USB stick and install Ubuntu into the unallocated space with manual partitioning.



- Boot from the USB stick (UEFI).
- Choose "Something else" and use the unallocated space.
- Create an ext4 root partition (/).
- Reuse the existing EFI partition (mount at /boot/efi, do not format).
- Install the boot loader to the main disk (e.g. /dev/nvme0n1).

4 Boot into Ubuntu

After installation, you can choose between Ubuntu and Windows.



- Remove the USB stick and reboot.
- You should see the GRUB menu with entries for Ubuntu and Windows.
- Select Ubuntu to continue.

Result

Dual-boot system with Windows + Ubuntu 24.04 LTS on a single disk.

5 Python Environment

Set up a dedicated Python virtual environment for the myCobot labs.

```
# Install Python and venv (usually preinstalled)
sudo apt install -y python3 python3-pip python3-venv

# Create a folder for the environments
mkdir -p ~/venvs

# Create and activate the virtual environment:
python3 -m venv ~/venvs/mycobot
source ~/venvs/mycobot/bin/activate

# Upgrade packaging tools
python -m pip install -U pip setuptools wheel

# (Optional) Install additional tools
python -m pip install -U colcon-common-extensions
rosdep

# Deactivate when done
deactivate
```

6 Install ROS 2 Jazzy

Install ROS 2 Jazzy from the official packages.

```
# 1. Update system
sudo apt update
sudo apt upgrade -y

# 2. Configure locale (if required)
sudo apt install -y locales
sudo locale-gen en_US en_US.UTF-8
sudo update-locale LC_ALL=en_US.UTF-8
LANG=en_US.UTF-8

# 3. Enable the ROS 2 repository
sudo apt install -y software-properties-common
sudo add-apt-repository universe -y

# 4. Add ROS 2 apt repository and key
sudo apt install -y curl
sudo curl -sSL https://raw.githubusercontent.com/ros/rosdistro/master/ros.key -o /usr/share/keyrings/ros-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/ros-archive-keyring.gpg] http://packages.ros.org/ros2/ubuntu $(. /etc/os-release && echo $UBUNTU_CODENAME) main" | \
sudo tee /etc/apt/sources.list.d/ros2.list

# 5. Install ROS 2 Jazzy (desktop)
sudo apt update
sudo apt install -y ros-jazzy-desktop
```

7 Source ROS and Test

Source ROS 2 and test the installation.

```
# Source ROS 2 automatically (for new terminals)
echo "source /opt/ros/jazzy/setup.bash" >> ~/.bashrc
source ~/.bashrc

# Install additional tools
sudo apt install -y python3-rosdep
python3-colcon-common-extensions
sudo rosdep init
rosdep update

# Test the installation
ros2 --version
ros2 run demo_nodes_cpp talker

# In a second terminal
ros2 run demo_nodes_cpp listener
```

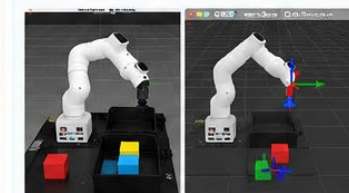
Successful Installation

- Ubuntu 24.04 LTS (dual-boot with Windows)
- Python virtual environment for the myCobot labs
- ROS 2 Jazzy (desktop) installed and sourced
- Ready for the practical lab with myCobot

8 Next Steps

You can now start working on the myCobot labs.

- ✓ Connect the myCobot to your PC (USB).
- ✓ Set up the ROS 2 workspace (colcon).
- ✓ Install the required Python packages in your virtual environment.
- ✓ Follow the lab instructions for simulation (Gazebo) and visualization (RViz).



Simulation in Gazebo

Visualization in RViz

ROS Overview



ROS Introduction – Overview

The Robot Operating System (ROS) – A flexible framework for robot software development



1 What is ROS?

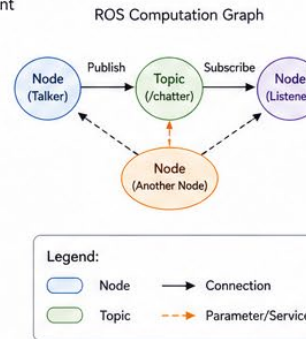
- Open-source framework for robot software development
- Provides tools, libraries, and conventions
- Enables modular, scalable and reusable code
- Supports a wide range of robots, sensors and platforms



ROS is a middleware that provides communication, tools and services so you can focus on robot behavior.

2 ROS Architecture

- Distributed system of independent processes (nodes)
- Nodes communicate through a ROS computation graph
- Master (ROS 1) or DDS (ROS 2) handles discovery and communication

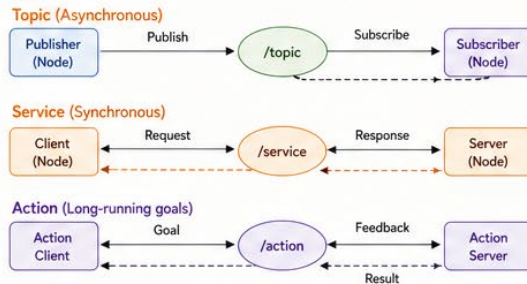


Key Idea: Decentralized, modular and loosely coupled system.

3 Core Concepts

- Node**
An executable process that performs computation.
- Topic**
Named bus over which nodes exchange messages (asynchronous).
- Message**
Typed data structure transmitted over topics (e.g., sensor_msgs/LaserScan).
- Service**
Synchronous request/response communication between nodes.
- Parameter**
Configuration data stored on the parameter server (ROS 1) or each node (ROS 2).

4 Communication Patterns



5 ROS 1 vs. ROS 2

	ROS 1	ROS 2
Release	2010	2017
Middleware	TCPROS	DDS
Communication	Centralized (Master)	Decentralized
Real-time support	Limited	Improved
Security	None	Built-in (SROS2)
Platforms	Linux, macOS	Linux, Windows, macOS
Languages	C++, Python, etc.	C++, Python, etc.
Lifecycle management	No	Yes (Lifecycle Nodes)

ROS 2 is the future – better performance, reliability, security and scalability.

6 ROS Ecosystem

- Large ecosystem of packages and tools
- Works with simulation, hardware and cloud
- Strong community and documentation
- Widely used in research and industry



Packages are organized in repositories and released via ROS build farm.

Key Takeaways



ROS enables communication between independent modules (nnodes) using standardized interfaces.



It provides tools and conventions to build complex robotic systems faster and more reliably.



Start simple: create nodes, communicate over topics, use services/actions, and build your robot application.



A large community and ecosystem support you from learning to real-world deployment.



Focus: Understand the fundamentals of ROS to build, connect and control intelligent robot systems.

ROS Overview

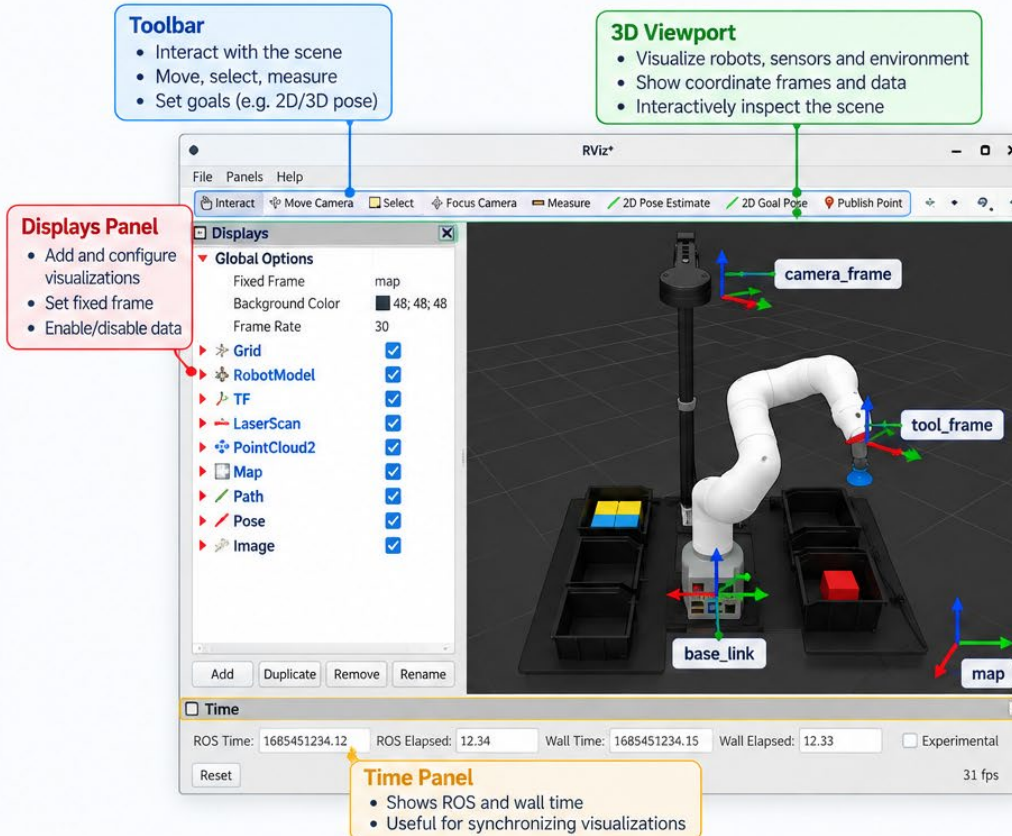
<http://www.ros.org>



Aspect	Description
Plumbing	Core communication infrastructure of ROS that provides message passing, service calls, actions, parameters, and transport mechanisms. It enables nodes to exchange data reliably and efficiently in a distributed system.
Tools	Development and runtime utilities that support building, debugging, visualization, monitoring, and deployment of robotic systems, such as build systems, launch tools, visualization tools, logging, and introspection utilities.
Capabilities	Reusable software libraries and packages that implement common robotic functionalities such as perception, navigation, localization, manipulation, mapping, and control. They provide ready-made building blocks for robot applications.
Community	Large open-source ecosystem of developers, researchers, and companies that contribute packages, documentation, tutorials, bug fixes, and long-term maintenance, enabling rapid innovation and shared best practices.

RViz – 3D Visualization in ROS 2

Visualize robots, sensors, maps and data in 3D



What is RViz?

- 3D visualization tool for ROS 2
- Displays robots, sensor data, coordinate frames, maps and planning results
- Helps with development, debugging and demonstration

Typical Use Cases



Visualize robot model (URDF)



Display sensor data (e.g. LiDAR, camera)



Show maps and localization (e.g. occupancy grid)



Visualize paths and goals (e.g. navigation)

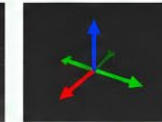
Common Display Types

RobotModel



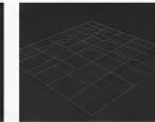
3D model from URDF

TF (Coordinate Frames)



Coordinate frames and transform tree

Grid



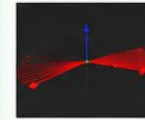
Reference grid in the fixed frame

Map



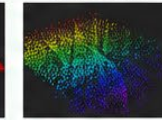
2D/3D map (e.g. from SLAM)

LaserScan



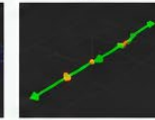
Laser scanner data

PointCloud2



3D point cloud data (e.g. depth camera, LiDAR)

Path



Planned or executed trajectory

Image



Camera image (e.g. RGB)

Typical Workflow



1. Start RViz

```
ros2 run rviz2 rviz2
```



2. Set Fixed Frame

e.g. map or base_link



3. Add Displays

RobotModel, TF, Map, LaserScan, ...



4. Configure Options

Set topics, colors, frames, styles



5. Visualize and Debug

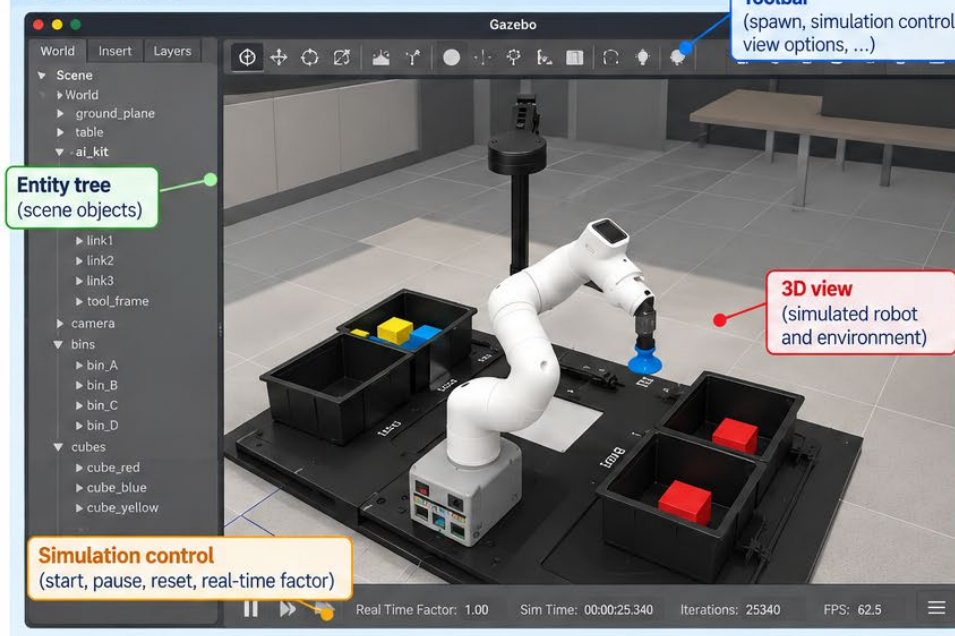
Inspect data and interact in 3D

Gazebo Physics Engine

Gazebo – Robot Simulation in ROS 2

Realistic 3D simulation of robots and environments

Gazebo GUI



What is Gazebo?

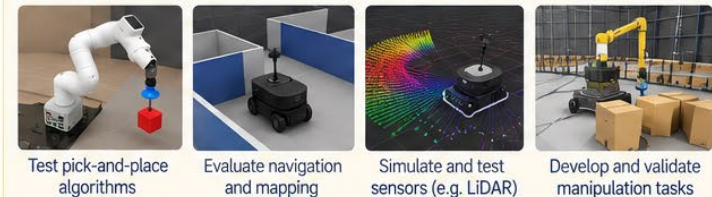
- Open-source 3D robot simulator for ROS 2
- Simulates robots, sensors and realistic environments
- Supports physics, dynamics, collisions and contacts
- Allows testing and development without real hardware
- Tightly integrated with ROS 2 (publish/subscribe, services)



Key Features

Realistic physics	Sensor simulation	Various robots	Flexible environments
• rigid bodies • collisions • contacts • gravity	• camera (RGB/Depth) • LiDAR • IMU • force/torque, ...	• support for URDF • mobile robots • manipulators • custom models	• indoor/outdoor worlds • objects and obstacles • lighting and materials • easy world editing

Typical Use Cases



World Description (SDF / URDF)



URDF (robot model, simplified)

```
<robot name="mycobot">
  <link name="base_link"> ... </link>
  <joint name="joint1" type="revolute">
    <parent link="base_link"/>
    <child link="link1"/>
    <origin xyz="0 0 0.1"/>
    ...
  </joint>
  ...
</robot>
```



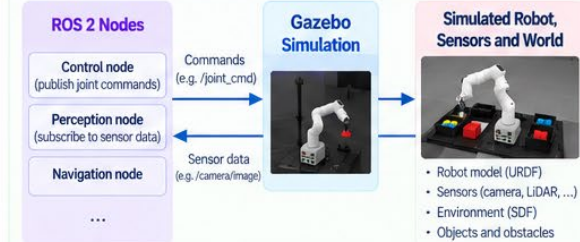
SDF (environment, simplified)

```
<world name="ai_kit_world">
  <include>
    <uri>model://ground_plane</uri> </include>
    <uri>model://table</uri> </include>
    <uri>model://bin_A</uri> ... </model>
    <uri>model://bin_B</uri> ... </model>
    <uri>model://bin_C</uri> ... </model>
    <uri>model://bin_D</uri> ... </model>
    <uri>model://mycobot</uri> </include>
  </include>
</world>
```

Simulation Workflow

- 1 **Model the robot (URDF)**
Load robot model (e.g. myCobot 320)
- 2 **Create the environment (SDF)**
Add ground, table, bins, objects
- 3 **Launch Gazebo**
Start simulation with ROS 2 launch file
- 4 **Run ROS 2 nodes**
Control the robot and read sensor data
- 5 **Test and iterate**
Develop and validate algorithms in simulation

Integration with ROS 2

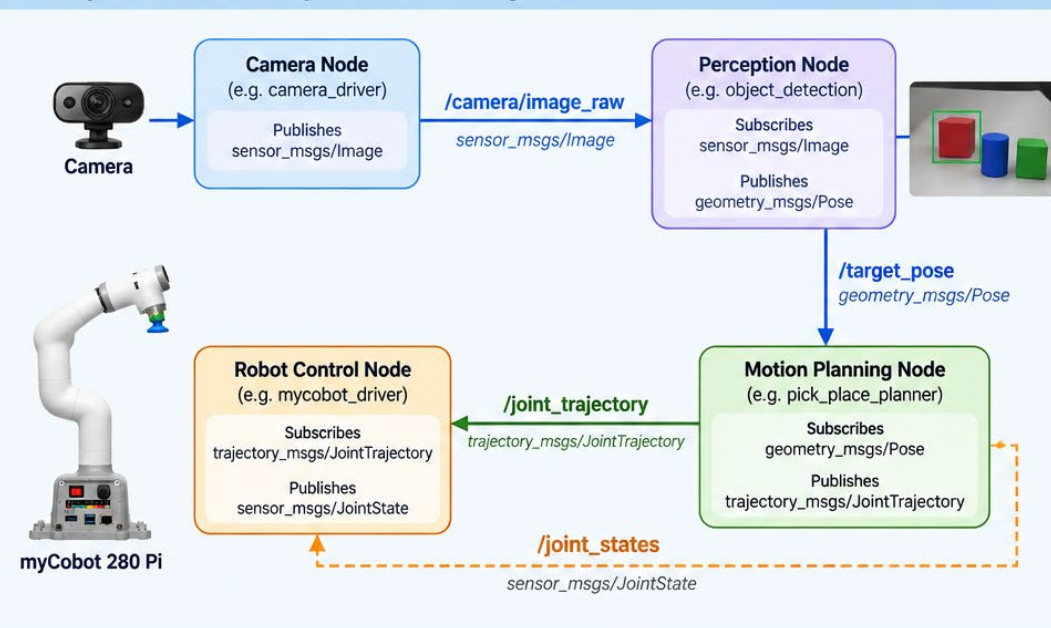


ROS Nodes, Topics & Messages

ROS 2 – Nodes, Topics and Messages

Communication in a distributed robotic system

Example: Mobile Manipulation with myCobot



1 Nodes

- A node is an executable process that performs a specific task (e.g. sensor driver, perception, planning, control).
- Nodes can publish and/or subscribe to topics.
- Multiple nodes run in parallel and communicate via the ROS 2 middleware.

2 Topics

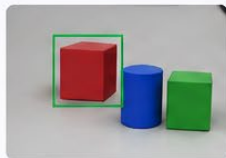
- A topic is a named communication channel for asynchronous message exchange (publish/subscribe).
- Nodes can publish messages to a topic.
- Other nodes can subscribe to the topic to receive the messages.
- Many nodes can publish to a topic and many nodes can subscribe to a topic.

3 Messages

- A message defines the data structure that is transmitted on a topic (e.g. `sensor_msgs/Image`, `geometry_msgs/Pose`).
- Each message type has a fixed set of fields with defined data types.
- Messages are defined in ROS 2 interface packages (e.g. `sensor_msgs`, `geometry_msgs`, `trajectory_msgs`).

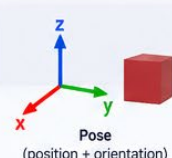
Example Message Types

Image message (`sensor_msgs/Image`)



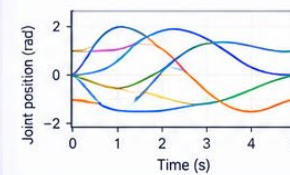
```
Header header
uint32 height
uint32 width
string encoding
uint8[] data
```

Pose message (`geometry_msgs/Pose`)



```
Point position
float64 x
float64 y
float64 z
Quaternion orientation
float64 x
float64 y
float64 z
float64 w
```

Joint trajectory message (`trajectory_msgs/JointTrajectory`)

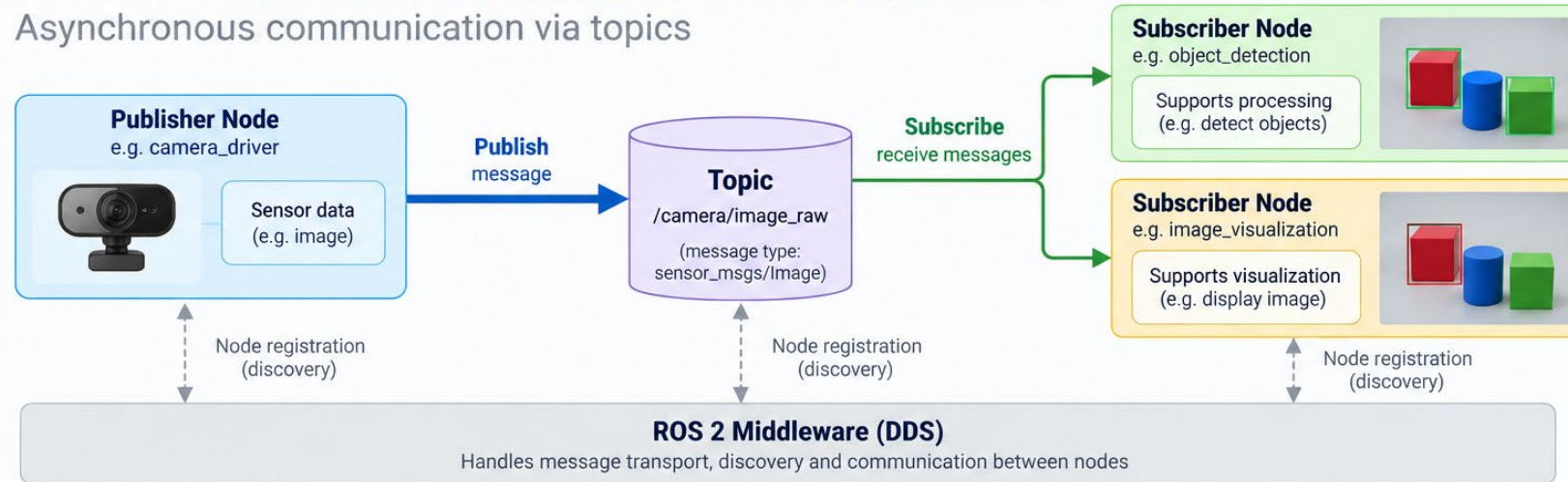


```
Header header
string[] joint_names
JointTrajectoryPoint[] points
float64[] positions
float64[] velocities
float64[] accelerations
builtin_interfaces/Duration
time_from_start
```

ROS Publisher Subscriber Architecture

ROS 2 – Publisher–Subscriber Architecture

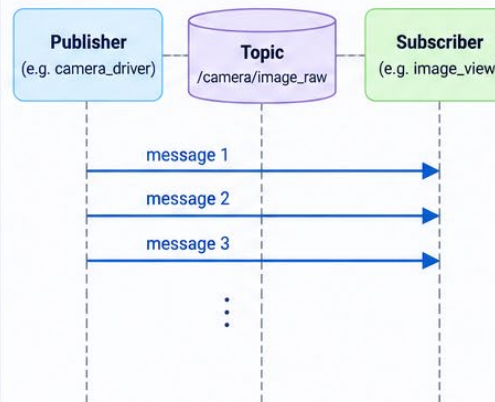
Asynchronous communication via topics



1. Key Concepts

- 1 Node** An executable process that performs a specific task (e.g. sensor driver, perception, control). Nodes can publish and/or subscribe to topics.
- 2 Topic** A named communication channel for asynchronous message exchange. Multiple publishers and multiple subscribers can use the same topic.
- 3 Message** Defines the data structure transmitted on a topic (e.g. sensor_msgs/Image, geometry_msgs/Twist).
- 4 Publisher** Sends messages to a topic.
- 5 Subscriber** Receives messages from a topic.

2. Message Flow



3. Example: Python Code (rclpy)

Publisher Node (publishes images)

```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image

class CameraNode(Node):
    def __init__(self):
        super().__init__('camera_driver')
        self.publisher = self.create_publisher(
            Image, '/camera/image_raw', 10)
        # ... read image and publish
        self.publisher.publish(img_msg)
```

Subscriber Node (receives images)

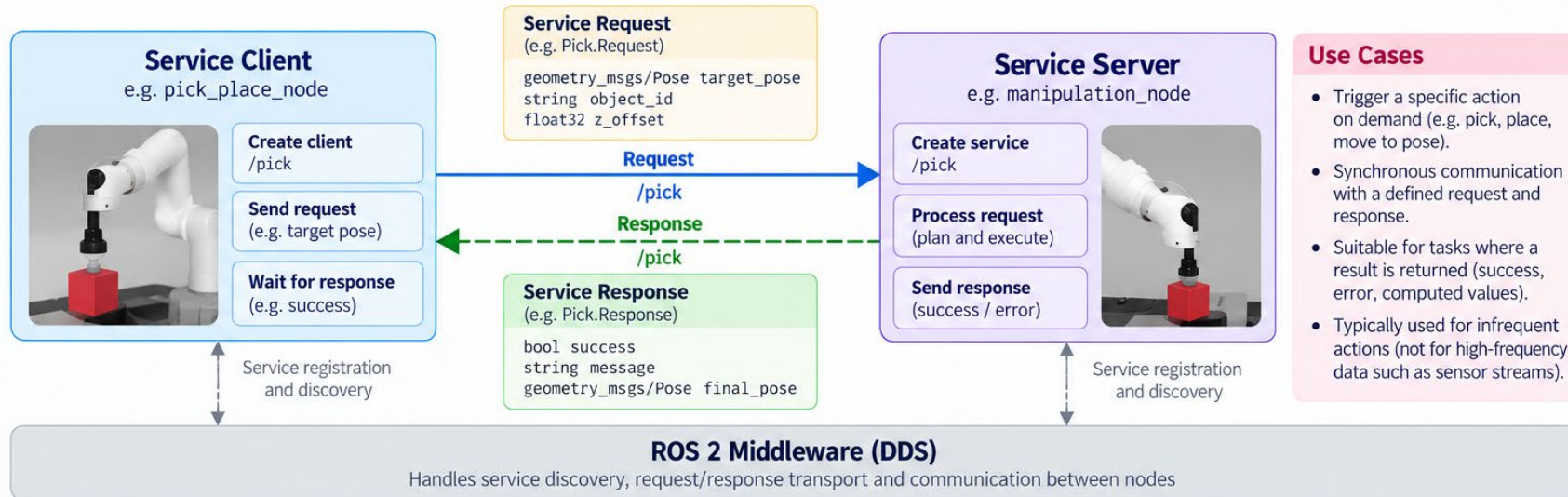
```
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image

class ImageViewNode(Node):
    def __init__(self):
        super().__init__('image_view')
        self.subscription = self.create_subscription(
            Image, '/camera/image_raw',
            self.image_callback, 10)

    def image_callback(self, msg):
        # process or display the received image
        pass
```

ROS 2 – Communication via Services

Synchronous request-response communication



1. Key Concepts

- Service**
A communication interface with a defined request and response message type.
- Service Server**
Provides a named service and processes incoming requests.
- Service Client**
Sends a request to a service and waits for the response.
- Synchronous**
The client blocks (or uses a future/callback) until the response is received.

2. Service Message Definition

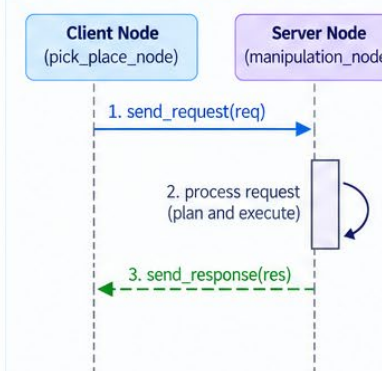
Example: Pick.srv

```
# Request
geometry_msgs/Pose target_pose
string object_id
float32 z_offset

---

# Response
bool success
string message
geometry_msgs/Pose final_pose
```

3. Communication Sequence



4. Example Usage

Command Line (ROS 2 CLI)

```
# Call the service /pick
ros2 service call /pick my_interfaces/srv/Pick
{target_pose: {position: {x: 0.3, y: 0.0, z: 0.1},
orientation: {x: 0.0, y: 0.0, z: 0.0, w: 1.0}},
object_id: "red", z_offset: 0.02}
```

Python Client Example (rclpy)

```
import rclpy
from rclpy.node import Node
from my_interfaces.srv import Pick

class PickClient(Node):
    ...
    req = Pick.Request()
    req.target_pose = target_pose
    req.object_id = 'red'
    req.z_offset = 0.02
    future = self.client.call_async(req)
    rclpy.spin_until_future_complete(self, future)
    res = future.result()
    print(f"Success: {res.success}, Message: {res.message}")
```

ROS Actions

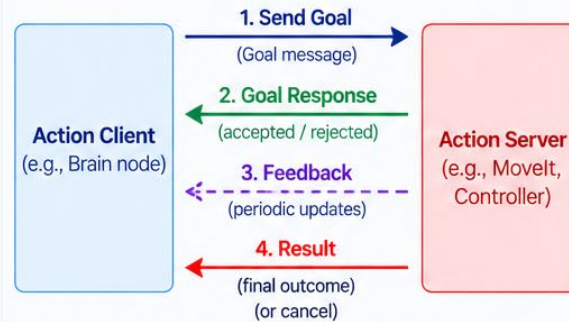
ROS 2 Actions

Asynchronous, goal-driven communication for long-running tasks

1. What is a ROS Action?

- A ROS 2 action is used for **long-running tasks** that provide **feedback** and return a **result**.
- It follows a **goal-driven, asynchronous** communication pattern.
- The client sends a goal, the server executes the task, periodically publishes **feedback**, and finally returns a result.
- Clients can **cancel** a goal at any time.

2. Communication Pattern



3. Action Interface Definition

An action is defined by a .action file, which specifies three message types: Goal, Result, and Feedback.

```
# example: MoveToPose.action
# Goal message: desired target
geometry_msgs/Pose target_pose
float64 speed
--- # separator
# Result message: final outcome
bool success
string message
--- # separator
# Feedback message: intermediate updates
float64 progress # 0.0 ... 1.0
string status
```

4. Action Topics and Services

An action uses a combination of topics and services with a common name, e.g., /move_action.

Name (for action /move_action)	Type / Purpose
/move_action/goal (service)	Send goal to server
/move_action/cancel (service)	Cancel an active goal
/move_action/get_result (service)	Request final result
/move_action/feedback (topic)	Feedback messages
/move_action/status (topic)	Status of all goals (e.g., pending, active, succeeded)
/move_action/result (topic)	Result messages (published when finished)

5. Example: MoveIt Execute Trajectory

MoveIt provides the action /move_action (moveit_msgs/action/MoveGroup) to execute planned motions.

```
# Goal (moveit_msgs/action/MoveGroup)
moveit_msgs/MotionPlanRequest
pipeline_id: ""
planner_id: ""
...

# Result
moveit_msgs/MoveItErrorCodes error_code
trajectory_msgs/JointTrajectory trajectory

# Feedback
moveit_msgs/MoveItErrorCodes error_code
float64 planning_time
float64 fraction # planned fraction
```

6. Usage Example (Python)

Send a goal, receive feedback, and get the result using an action client.

```
from rclpy.action import ActionClient
from moveit_msgs.action import MoveGroup

client = ActionClient(node, MoveGroup,
                     '/move_action')

# send goal
send_goal_future = client.send_goal_async(
    goal_msg,
    feedback_callback=feedback_cb)

# handle result
def feedback_cb(feedback_msg):
    print(f"Progress: {feedback_msg.fraction:.2f}")

result_future = send_goal_future.get_result_async()
```

7. Typical Use Cases

- Robot motion execution (e.g., MoveIt)
- Grasping or pick-and-place sequences
- Navigation tasks (e.g., NavigateToPose in Nav2)
- Any task that takes noticeable time, provides meaningful intermediate feedback, and can be canceled
- Not suited for short, instantaneous requests (use services instead)

MoveIt in ROS 2 – Overview

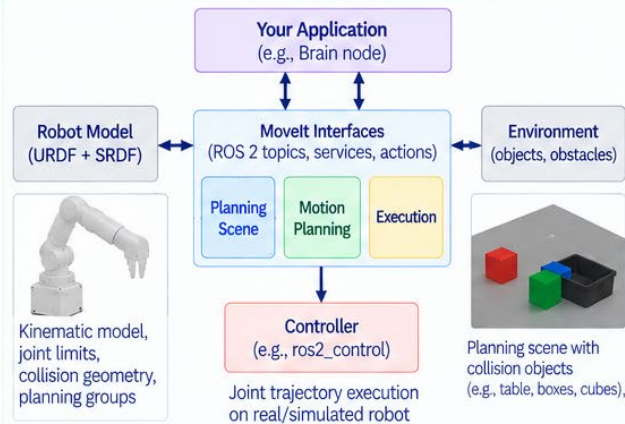
Motion planning, collision checking and execution for robotic manipulators

1. What is MoveIt?

- MoveIt is a ROS package for motion planning, collision checking, and manipulation.
- Provides high-level interfaces to plan and execute robot motions.
- Uses a robot model (URDF/SRDF) and a planning scene with the environment and objects.
- Supports different planners (e.g., OMPL) and can work with both simulation and real robots.

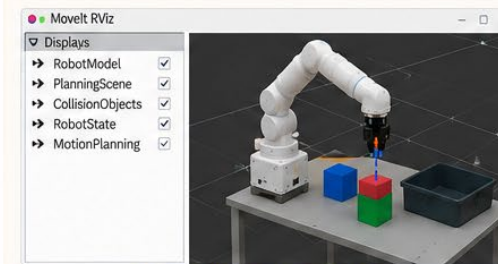


2. MoveIt Architecture in ROS 2



3. Planning Scene

- Represents the robot and its environment for collision checking and planning.
- Contains the robot model, attached objects and external collision objects.
- Can be updated dynamically (e.g., add/remove objects).



Example: Planning scene in RViz

4. Motion Planning

- Computes a collision-free path from a start to a goal pose or joint configuration.
- Uses sampling-based planners (e.g., OMPL: RRT, PRM, RRTConnect, etc.).
- Plans in joint space or Cartesian space.
- Considers joint limits and collision objects from the planning scene.

Joint-space planning



Plan a path in joint space

Cartesian path planning



Plan a path for the end-effector in Cartesian space

5. MoveIt Interfaces (ROS 2)

Topics (Planning Scene)

- /collision_object (moveit_msgs/msg/CollisionObject)
- /attached_collision_object (moveit_msgs/msg/AttachedCollisionObject)

Services

- /compute_cartesian_path (moveit_msgs/srv/GetCartesianPath)
- /get_planning_scene (moveit_msgs/srv/GetPlanningScene)
- /apply_planning_scene (moveit_msgs/srv/ApplyPlanningScene)

Actions

- /move_action (moveit_msgs/action/MoveGroup)
- /execute_trajectory (moveit_msgs/action/ExecuteTrajectory)

6. Example Workflow: Pick and Place

1. Add environment objects to the planning scene (e.g., table, boxes)
2. Plan approach to grasp pose (e.g., Cartesian path or joint goal)
3. Execute trajectory to pre-grasp pose
4. Move down and grasp (e.g., activate gripper)
5. Plan and execute motion to place pose
6. Release object and retract



Approach



Grasp



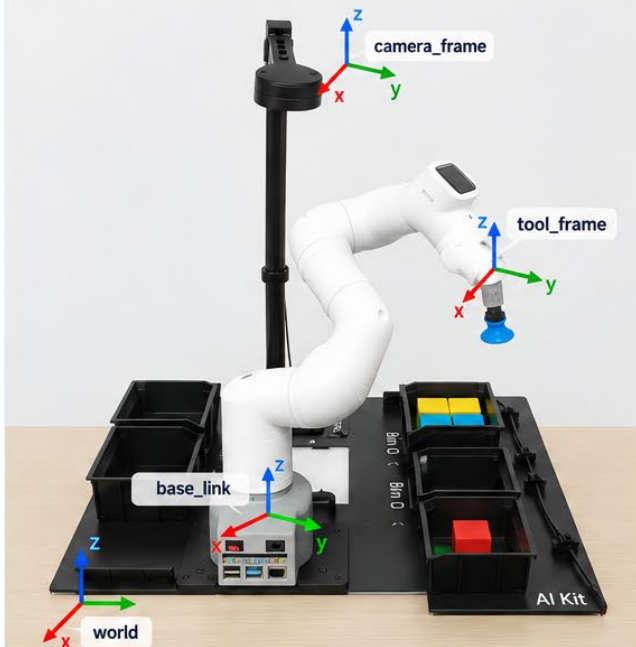
Place

ROS Transformation Tree

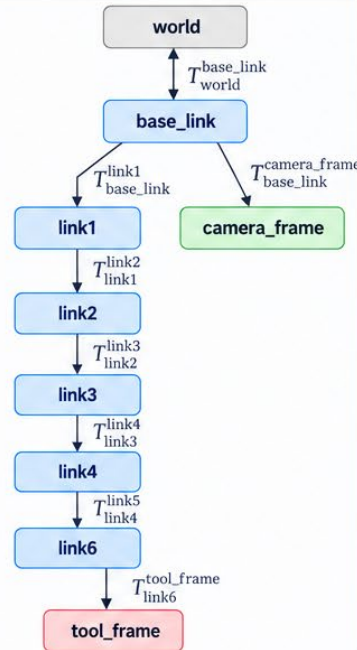
ROS 2 – Transformation Tree (TF)

Coordinate frames and transformations in a robotic system

Example: myCobot 320 with Camera and Base



TF Tree Structure



Key Points

- 1 Coordinate Frames**
 - Each frame defines a local coordinate system (x, y, z).
 - Frames are attached to robot links, sensors, tools or the world.
 - The transform specifies the position and orientation between two frames.
- 2 TF Tree**
 - Forms a tree structure with a single root frame (e.g. world or map).
 - Each frame (except the root) has exactly one parent.
 - Transforms between any two frames can be computed by chaining the transforms along the tree.
- 3 Transform Representation**
 - A transform consists of a translation (x, y, z) and a rotation (quaternion or RPY).
 - Denoted as T_{parent}^{child} (pose of child frame w.r.t. parent frame).
- 4 Common Frames in Robotics**
 - world / map:** global reference frame (e.g. motion capture, map).
 - base_link:** robot base frame.
 - link1 ... link6:** robot arm link frames.
 - tool_frame:** end-effector / gripper frame.
 - camera_frame:** RGB/Depth camera frame.

Transform Example

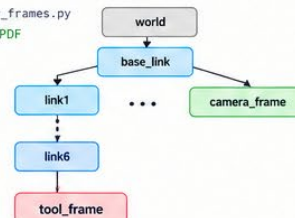
Transform from base_link to tool_frame:

$$T_{base_link}^{tool_frame} = T_{base_link}^{link1} T_{link1}^{link2} \dots T_{link6}^{tool_frame}$$

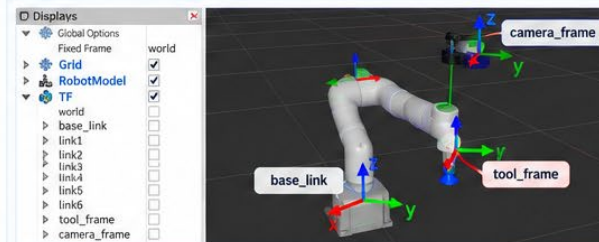


View the TF Tree

```
# Show all frames
ros2 run tf2_tools view_frames.py
# Open the generated PDF
evince frames.pdf
```



Visualization in RViz



Serial Communication between myCobot and ROS

Serial Communication: myCobot (Elephant) ↔ ROS

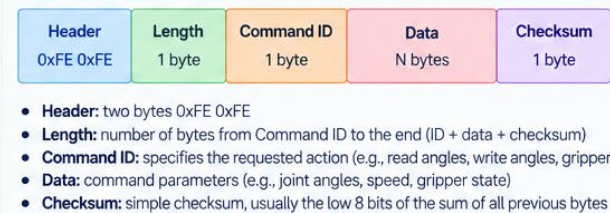
How commands and data are exchanged between ROS and the myCobot 280

1. System Overview



2. Communication Protocol

2.1 Frame Structure



2.2 Example Frames (simplified)

Read all joint angles (from robot to ROS)



Write joint angles (ROS to robot)

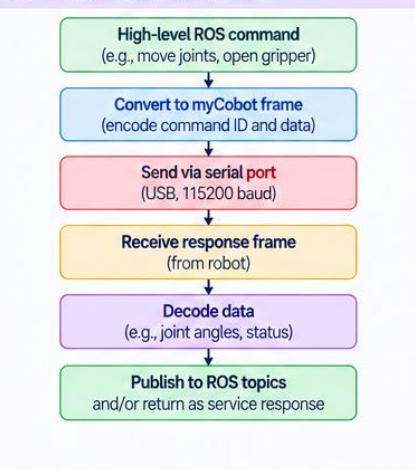


3. Common Command IDs

Command ID (hex)	Function	Data (example)
0x20	Read all joint angles	-
0x21	Read single joint angle	Joint ID (1 byte)
0x22	Write joint angles	6 × int16 (angles)
0x23	Write single joint angle	Joint ID + angle
0x24	Set joint speed	Joint ID + speed
0x26	Read robot status	-
0x28	Control gripper	Open/Close, speed
0x29	Read gripper status	-

i The exact command set may vary slightly depending on the firmware version. For details, refer to the official myCobot 280 communication protocol documentation.

4. ROS Node Workflow



5. Example: Send Joint Angles (Python)

```
# Example: send joint angles to myCobot 280
import serial
import struct

# Open serial port
ser = serial.Serial('/dev/ttyUSB0', 115200, timeout=1)

# Desired joint angles (degrees)
angles = [0, -30, 60, 0, 45, 0]
speed = 50 # 1-100

# Convert to protocol format (int16, 0.1 deg)
angle_data = b''.join([struct.pack('<h', int(a * 10)) for a in angles])
speed_data = struct.pack('<B', speed)

# Build frame: FE FE | len | cmd | data | checksum
cmd_id = 0x22
payload = angle_data + speed_data
length = 1 + len(payload) + 1
frame = bytearray([0xFE, 0xFE, length, cmd_id]) + payload
checksum = sum(frame[2:]) & 0xFF
frame.append(checksum)

# Send to robot
ser.write(frame)
```

6. Example: Read Joint Angles (Python)

```
# Example: read all joint angles from myCobot 280
import serial
import struct

# Open serial port
ser = serial.Serial('/dev/ttyUSB0', 115200, timeout=1)

# Build and send frame: FE FE 02 20 22
frame = bytes([0xFE, 0xFE, 0x02, 0x20, 0x22])
ser.write(frame)

# Read response
response = ser.read(100)

# Check header and command ID
if len(response) >= 15 and response[0:2] == b'\xFE\xFE':
    data = response[4:16] # 6 × int16 angles (0.1 deg)
    angles = [struct.unpack('<h', data[i:i+2])[0] / 10.0
               for i in range(0, 12, 2)]
    print('Joint angles [deg]:', angles)
else:
    print('Invalid response frame')
```

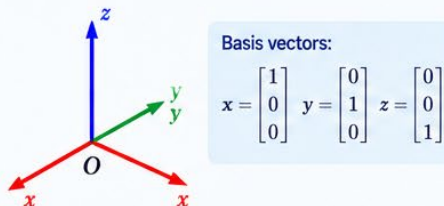
Spatial Transformations

Spatial Transformations in Robotics

Rotation matrices, homogeneous transformations, and frame transformations

1. Coordinate Frames and Pose

A coordinate frame $\{F\}$ is defined by an origin and three orthonormal basis vectors:



Pose of a frame $\{B\}$ with respect to $\{A\}$:

- Position: $p \in \mathbb{R}^3$ (origin of $\{B\}$ expressed in $\{A\}$)
- Orientation: $R \in SO(3)$ (rotation of $\{B\}$ w.r.t. $\{A\}$)

2. Rotation Matrices

A rotation matrix $R \in SO(3)$ represents the orientation of a frame. It is orthonormal with determinant 1:

$$R^T R = I, \quad \det(R) = 1$$

Rotations about coordinate axes:

$$R_x(\theta) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\theta & -s_\theta \\ 0 & s_\theta & c_\theta \end{bmatrix}$$

$$R_y(\theta) = \begin{bmatrix} c_\theta & 0 & s_\theta \\ 0 & 1 & 0 \\ -s_\theta & 0 & c_\theta \end{bmatrix}$$

$$R_z(\theta) = \begin{bmatrix} c_\theta & -s_\theta & 0 \\ s_\theta & c_\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$c_\theta = \cos(\theta), \quad s_\theta = \sin(\theta)$$

3. Homogeneous Transformation

A rigid body transformation (pose) is represented by a 4×4 homogeneous transformation matrix:

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} \in SE(3)$$

- $R \in SO(3)$: rotation matrix
- $p \in \mathbb{R}^3$: position vector
- $T \in SE(3)$: special Euclidean group (rigid body transformations)

Inverse transformation:

$$T^{-1} = \begin{bmatrix} R^T & -R^T p \\ 0 & 1 \end{bmatrix}$$

Composition (concatenation):

$$T_{AC} = T_{AB} T_{BC}$$

4. Transforming Points and Vectors

Transforming a point:

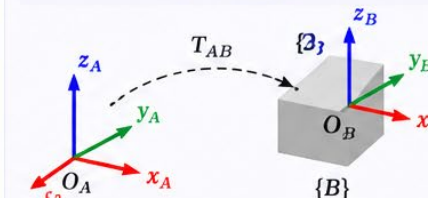
A point $p_B = [x \ y \ z]^T$ in frame $\{B\}$ is expressed in frame $\{A\}$ as:

$$p_A = T_{AB} \begin{bmatrix} p_B \\ 1 \end{bmatrix} = R_{AB} p_B + p_{AB}$$

Transforming a vector (direction):

A vector v_B in frame $\{B\}$ transforms as:

$$v_A = R_{AB} v_B$$



5. Euler Angles (Roll-Pitch-Yaw)

A common parameterization of a rotation matrix is given by roll (ϕ), pitch (θ), and yaw (ψ):

$$R = R_z(\psi) R_y(\theta) R_x(\phi)$$

$$R = \begin{bmatrix} c_\psi c_\theta & c_\psi s_\theta s_\phi - s_\psi c_\phi & c_\psi s_\theta c_\phi + s_\psi s_\phi \\ s_\psi c_\theta & s_\psi s_\theta s_\phi + c_\psi c_\phi & s_\psi s_\theta c_\phi - c_\psi s_\phi \\ -s_\theta & c_\theta s_\phi & c_\theta c_\phi \end{bmatrix}$$

$$c_\psi = \cos(\psi), \quad s_\psi = \sin(\psi)$$

Notes:

- Different conventions (e.g., ZYX, XYZ) exist.
- Euler angles are intuitive but can have singularities (gimbal lock).
- Alternatively, use rotation matrices or quaternions.

6. Example: Transform a Point

A point $p_B = [0.1, 0.0, 0.0]^T$ m in frame $\{B\}$. Frame $\{B\}$ is rotated by 30° around z and translated by $p_{AB} = [0.4, 0.2, 0.1]^T$ m.

Transformation:

$$R = R_z(30^\circ) = \begin{bmatrix} 0.866 & -0.5 & 0 \\ 0.5 & 0.866 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$T_{AB} = \begin{bmatrix} R & p_{AB} \\ 0 & 1 \end{bmatrix}$$

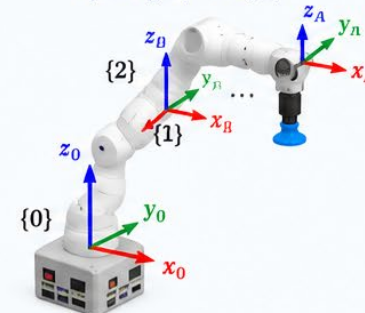
Point in frame $\{A\}$:

$$p_A = R p_B + p_{AB} = \begin{bmatrix} 0.486 \\ 0.250 \\ 0.100 \end{bmatrix} \text{ m}$$

7. Frame Transformations in a Chain

For a serial robot with frames $\{0\}, \{1\}, \dots, \{n\}$ the transformation from base to end-effector is:

$$T_0^n = T_0^1 T_1^2 \dots T_{n-1}^n$$



8. Key Takeaways

- A rigid body pose is represented by a homogeneous transformation matrix $T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} \in SE(3)$.
- Rotation matrices are orthonormal with determinant 1 and describe orientation.
- Points are transformed with the full homogeneous matrix, vectors only with the rotation part.
- Transformations can be composed along a kinematic chain.
- Euler angles are an intuitive representation, but quaternions or rotation matrices are often preferred in practice.

Assignment Spatial Transformations

Spatial Transformations Assignment – Overview

Representations of rotations and rigid body transformations in 3D and practical implementation in Python and ROS 2

1. Goal and Learning Objectives

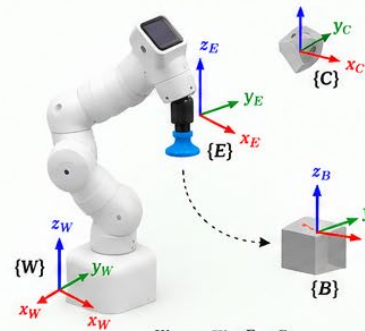
Goal

Understand and apply mathematical representations of spatial transformations (rotations and translations) and use them to model and visualize robotic systems (myCobot 280).

You will learn to:

- Represent, compose, and convert between different rotation parameterizations (rotation matrices, Euler angles, RPY, angle-axis, quaternions).
- Construct and use homogeneous transformation matrices.
- Transform points and vectors between coordinate frames.
- Model kinematic structures (rigid body tree) and visualize them in Python and ROS 2 (RViz2).
- Implement and test transformations using NumPy, SpatialMath, and the Robotics Toolbox for Python.

2. Coordinate Frames and Pose



$$T_B^W = T_E^W T_C^E T_B^C$$

- {W} World / robot base frame
- {E} End-effector (tool) frame
- {C} Camera frame
- {B} Object frame

3. Rotation Matrices

A rotation matrix $R \in SO(3)$ is orthonormal with determinant 1:

$$R^T R = I, \quad \det(R) = 1$$

Basic rotations about coordinate axes:

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & c_\alpha & -s_\alpha \\ 0 & s_\alpha & c_\alpha \end{bmatrix}$$

$$R_y(\beta) = \begin{bmatrix} c_\beta & 0 & s_\beta \\ 0 & 1 & 0 \\ -s_\beta & 0 & c_\beta \end{bmatrix}$$

$$R_z(\gamma) = \begin{bmatrix} c_\gamma & -s_\gamma & 0 \\ s_\gamma & c_\gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$c_\theta = \cos(\theta), \quad s_\theta = \sin(\theta)$$

4. Euler Angles and RPY

Roll-Pitch-Yaw (RPY)

Common ZYX convention (intrinsic rotations):

$$R = R_z(\psi) R_y(\theta) R_x(\phi)$$



Rotation Properties

- Different conventions exist: XYZ, ZYX, intrinsic/extrinsic.
- Euler angles are intuitive but can have singularities (gimbal lock).
- Order of rotations matters.
- Use rotation matrices or quaternions for robust computations if needed.

5. Angle-Axis Representation

A rotation by angle θ around a unit axis $\hat{u} = [u_x, u_y, u_z]^T$ is given by Rodrigues' formula:

$$R = I + \sin(\theta)[\hat{u}]_\times + (1 - \cos(\theta))[\hat{u}]_\times^2$$

Skew-symmetric matrix:

$$[\hat{u}]_\times = \begin{bmatrix} 0 & -u_z & u_y \\ u_z & 0 & -u_x \\ -u_y & u_x & 0 \end{bmatrix}$$

6. Quaternions

A unit quaternion $q = [q_w, q_x, q_y, q_z]^T$ represents a rotation without singularities.

- Norm: $\|q\| = 1$
- Conversion to rotation matrix:

$$R(q) = \begin{bmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_z q_w) & 2(q_x q_z + q_y q_w) \\ 2(q_x q_y + q_z q_w) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_x q_w) \\ 2(q_x q_z - q_y q_w) & 2(q_y q_z + q_x q_w) & 1 - 2(q_x^2 + q_y^2) \end{bmatrix}$$

Useful for:

- stable rotation composition
- interpolation (SLERP)
- avoiding gimbal lock

7. Homogeneous Transformation

A rigid body transformation (pose) is represented by a 4×4 matrix:

$$T = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} \in SE(3)$$

- $R \in SO(3)$: rotation matrix
- $p \in \mathbb{R}^3$: translation vector
- $T \in SE(3)$: special Euclidean group

Inverse and composition:

$$T^{-1} = \begin{bmatrix} R^T & -R^T p \\ 0 & 1 \end{bmatrix}$$

$$T_{AC} = T_{AB} T_{BC}$$

Transforming points and vectors:

Point:

$$p_A = T_{AB} \begin{bmatrix} p_B \\ 1 \end{bmatrix} = R_{AB} p_B + p_{AB}$$

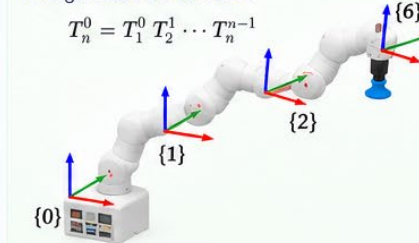
Vector (direction):

$$v_A = R_{AB} v_B$$

8. Kinematic Chain and Rigid Body Tree

A serial manipulator is modeled by a chain of homogeneous transformations:

$$T_n^0 = T_1^0 T_2^1 \dots T_n^{n-1}$$



9. Practical Tasks in the Assignment

Install and set up environment
(Ubuntu 24.04, ROS 2 Jazzy,
Python, myCobot package)

Implement basic rotations
and conversions
(Numpy/SpatialMath)

Work with homogeneous
transformations and
frame transformations

Model the myCobot 280
kinematic chain and visualize
in RViz2

10. Key Takeaways

- ✓ Spatial transformations describe positions and orientations in 3D using rotation matrices, Euler angles, angle-axis, quaternions, and homogeneous transforms.
- ✓ They are essential for robot kinematics, sensor calibration, and motion planning.
- ✓ You will implement these concepts in Python and visualize them in ROS 2 (RViz2) using the myCobot 280.

Direct Kinematics Robot Arm

Direct Kinematics of a Robot Arm

From joint angles to the pose of the end-effector

1. Problem Statement

Direct kinematics (forward kinematics) computes the position and orientation (pose) of the end-effector from the given joint variables.

Input:

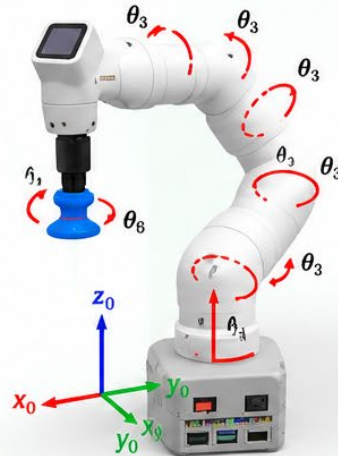
Joint angles $q = [\theta_1, \theta_2, \dots, \theta_6]$

Output:

End-effector pose $T_0^6 \in SE(3)$

- Position $p = [x, y, z]^T$
- Orientation R (e.g. as rotation matrix, Euler angles or quaternion)

2. Robot Model and Frames



3. Kinematic Chain (DH Parameters)

Each link is described by a homogeneous transformation using Denavit–Hartenberg (DH) parameters.

i	a_{i-1}	α_{i-1}	d_i	θ_i
1	a_0	α_0	d_1	θ_1
2	a_1	α_1	d_2	θ_2
3	a_2	α_2	d_3	θ_3
4	a_3	α_3	d_4	θ_4
5	a_4	α_4	d_5	θ_5
6	a_5	α_5	d_6	θ_6

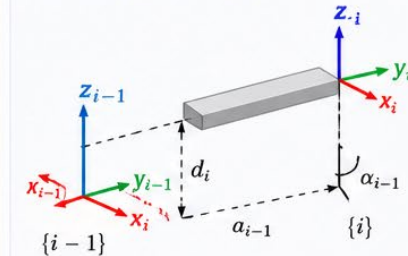
Homogeneous transformation from frame $\{i-1\}$ to $\{i\}$:

$$A_i = R_z(\theta_i)T_z(d_i)T_x(a_{i-1})R_x(\alpha_{i-1})$$

4. Homogeneous Transformation

$$A_i = \begin{bmatrix} c\theta_i & -s\theta_i c\alpha_{i-1} & s\theta_i s\alpha_{i-1} & a_{i-1} c\theta_i \\ s\theta_i & c\theta_i c\alpha_{i-1} & -c\theta_i s\alpha_{i-1} & a_{i-1} s\theta_i \\ 0 & s\alpha_{i-1} & c\alpha_{i-1} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$c\theta = \cos(\theta), \quad s\theta = \sin(\theta)$$



5. Forward Kinematics

The overall transformation from the base $\{0\}$ to the end-effector $\{6\}$ is the product of all link transformations:

$$T_0^6 = A_1 A_2 A_3 A_4 A_5 A_6 = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}$$

- $R \in SO(3)$: rotation matrix of the end-effector
- $p = [x, y, z]^T$: position of the end-effector in the base frame

End-effector pose:

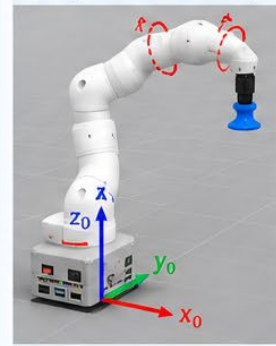
Position: $p = [x, y, z]^T$

Orientation: R or (roll, pitch, yaw) or quaternion (q_w, q_x, q_y, q_z)

6. Example: myCobot 280

Joint angles (example)

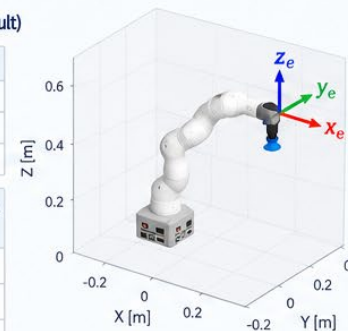
$$q = [0, -30, 60, 0, 45, 0]^\circ$$



End-effector pose (result)

Position p [m]	
x	0.28
y	0.05
z	0.35

Orientation (roll, pitch, yaw) [°]	
roll	180.0
pitch	0.0
yaw	45.0



7. Applications

- Compute the end-effector position for a given joint configuration.
- Visualize the robot pose in simulation (e.g. in RViz or MATLAB).
- Check reachability and workspace.
- Use forward kinematics to verify and analyze motion commands.

Key Takeaways

- Direct kinematics maps joint angles to the end-effector pose.
- It is computed using a chain of homogeneous transformations (e.g. DH parameters).
- The result provides the position and orientation of the end-effector in the base frame, which is used for visualization, analysis and control.

Assignment Forward Kinematics

Forward Kinematics Assignment – myCobot 280

From joint angles to the end-effector pose and visualization in ROS 2

1. Goal and Overview

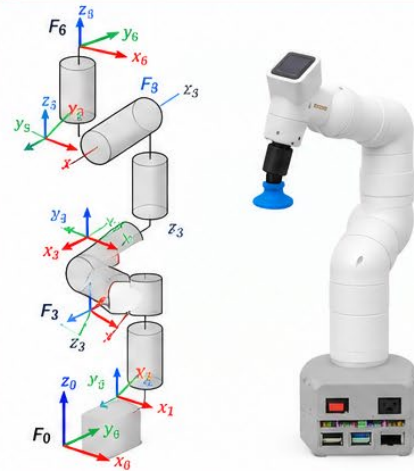
Goal

- Compute the forward kinematics of the 6-DOF myCobot 280.
- Map joint angles $q = [\theta_1, \dots, \theta_6]^T$ to the end-effector pose T_6^0 .
- Implement and visualize the kinematic model in Python and in ROS 2 (RViz2).

Tools

- Ubuntu 24.04 & ROS 2 Jazzy
- Python (Jupyter Notebook)
- NumPy, SpatialMath, Robotics Toolbox for Python
- myCobot ROS 2 package
- RViz2 for visualization

2. Robot Model and Joint Frames



3. Denavit–Hartenberg Parameters

i	a_{i-1} [m]	α_{i-1} [°]	d_i [m]	θ_i
1	0	0	d_1	θ_1
2	a_1	-90	0	θ_2
3	a_2	0	0	θ_3
4	0	90	d_4	θ_4
5	0	-90	0	θ_5
6	0	0	d_6	θ_6

Link transformation

$$A_i = R_z(\theta_i)T_x(d_i)T_x(a_{i-1})R_x(\alpha_{i-1})$$

Forward kinematics

$$T_6^0 = A_1 A_2 A_3 A_4 A_5 A_6 = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix}$$

4. End-Effector Pose

The end-effector pose T_6^0 describes the position and orientation of the tool frame {6} with respect to the base frame {0}.

$$T_6^0 = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} \in SE(3)$$

Position

$$p = [x, y, z]^T \text{ [m]}$$

Orientation

- Rotation matrix $R \in SO(3)$ or
- Roll-Pitch-Yaw (ϕ, θ, ψ) [°] or
- Quaternion (q_w, q_x, q_y, q_z)



5. Workflow of the Assignment

Set up development environment

Ubuntu 24.04, ROS 2 Jazzy, myCobot package

Define DH model

DH parameters and link transforms

Implement forward kinematics

Python (NumPy / SpatialMath / Robotics Toolbox)

Validate results

Test configurations, compare with provided tools

Visualize in ROS 2 (RViz2)

URDF model and joint states

6. Example: Compute End-Effector Pose (Python)

```
# Forward kinematics using DH parameters (example)
import numpy as np
from spatialmath import SE3
from roboticstoolbox import DHRobot, RevoluteDH

# Define DH parameters for myCobot 280 (example values)
# (Replace with the actual parameters from the lab)
dh_params = [
    [0.0, 0.0, 0.1035, 0.0],
    [0.0, -np.pi/2, 0.0, 0.0],
    [0.135, 0.0, 0.0, 0.0],
    [0.0, np.pi/2, 0.147, 0.0],
    [0.0, -np.pi/2, 0.0, 0.0],
    [0.0, 0.0, 0.074, 0.0]
]

robot = DHRobot([RevoluteDH(a, alpha, d, theta)
                 for a, alpha, d, theta in dh_params], name='myCobot280')

# Example joint configuration (degrees)
q_deg = [0, -30, 60, 0, 45, 0]
q = np.deg2rad(q_deg)
T06 = robot.fkine(q)
print(T06)
print('Position [m]:', T06.t)
print('RPY [deg]:', T06.rpy(order='xyz', unit='deg'))
```

Result: End-Effector Pose

Homogeneous transformation

$$T_6^0 = \begin{bmatrix} 0.71 & -0.71 & 0.00 & 0.28 \\ 0.71 & 0.71 & 0.00 & 0.05 \\ 0.00 & 0.00 & 1.00 & 0.35 \\ 0.00 & 0.00 & 0.00 & 1.00 \end{bmatrix}$$

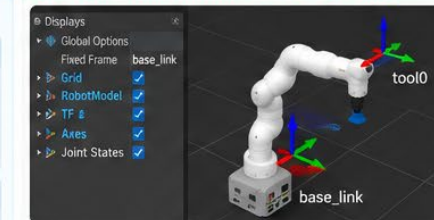
Position

$x = 0.28$ m
 $y = 0.05$ m
 $z = 0.35$ m

Orientation (RPY, xyz)

roll = 180.0°
pitch = 0.0°
yaw = 45.0°

7. Visualization in ROS 2 (RViz2)



8. What You Learn

- Model the kinematics of a 6-DOF robotic arm using DH parameters.
- Compute the end-effector pose from joint angles.
- Use Python and robotics libraries for kinematic modeling.
- Visualize and validate the model in ROS 2 (RViz2).
- Understand the mapping between joint space and task space.

Inverse Kinematics

Inverse Kinematics of Robotic Manipulators

From end-effector pose to joint angles

1. Problem Statement

Inverse kinematics (IK) computes the joint configuration $\mathbf{q} = [\theta_1, \dots, \theta_n]^T$ that realizes a desired end-effector pose

$$T_e^0 = \begin{bmatrix} \mathbf{R} & \mathbf{p} \\ 0 & 1 \end{bmatrix} \in SE(3)$$

Input:

Desired pose T_e^0 (position and orientation)



Output:

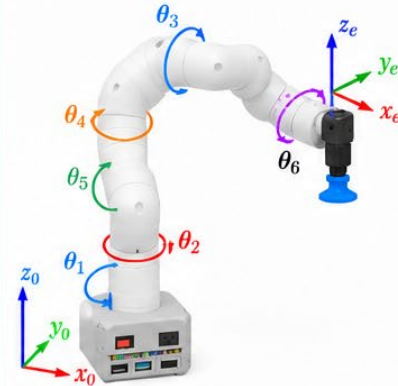
Joint angles $\mathbf{q} = [\theta_1, \dots, \theta_n]^T$ that achieve the pose

There can be zero, one, or multiple solutions. IK may require numerical methods and additional constraints.

2. Kinematic Model

Use the forward kinematics

$$T_e^0(\mathbf{q}) = T_1^0(q_1) T_2^1(q_2) \dots T_n^{n-1}(q_n)$$



3. IK Formulation

Find $\mathbf{q}$ such that

$$f(\mathbf{q}) = \log \left(T_e^0(\mathbf{q})^{-1} T_{e,d}^0 \right) = 0$$

Pose error (6D)

$$f(\mathbf{q}) = \begin{bmatrix} \mathbf{p}(\mathbf{q}) - \mathbf{p}_d \\ e_R(R(\mathbf{q}), R_d) \end{bmatrix} \in \mathbb{R}^6$$

- Position error: $\mathbf{p}(\mathbf{q}) - \mathbf{p}_d \in \mathbb{R}^3$
- Orientation error: $e_R \in \mathbb{R}^3$
 - e.g. axis-angle, rotation vector or quaternion-based error

4. Solution Methods

Analytical IK

- Closed-form solution for specific robot structures (e.g. 6R with spherical wrist)
- Fast and exact (if solution exists)

Numerical IK (Iterative)

- Minimize pose error, e.g. using Newton-Raphson, Levenberg-Marquardt (damped least squares)
- General method, works for many robots

Jacobian-based update

$$\Delta \mathbf{q} = \mathbf{J}^{\dagger}(\mathbf{q}) \mathbf{e}, \quad \mathbf{q}_{k+1} = \mathbf{q}_k + \Delta \mathbf{q}$$

- $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{6 \times n}$: geometric Jacobian
- $\mathbf{J}^{\dagger}$: pseudoinverse or damped pseudoinverse

5. Multiple Solutions and Constraints



A desired pose can have multiple IK solutions.

- Solution 1
- Solution 2
- Solution 3

Additional constraints

- Joint limits
- Collision avoidance
- Preferred configuration (e.g. closest to current $\mathbf{q}$)
- Task-specific constraints (e.g. end-effector orientation)

6. Example: Numerical IK (Python)

```
# Example: numerical IK using damped least squares
from roboticstoolbox import DHRobot, RevoluteDH
import numpy as np

# create robot model (example: myCobot 280 with DH parameters)
robot = DHRobot(dh_params, name='myCobot280')

# desired end-effector pose
T_des = SE3(0.3, 0.1, 0.2) * SE3.RPY([0, np.pi/2, 0])

# initial guess
q0 = np.zeros(6)

# solve IK (using toolbox function)
q_sol = robot.ikine_lm(T_des, q0=q0, mask=[1,1,1,1,1,1])

print("Joint angles [deg]:", q_sol.q * 180/np.pi)
print("Achieved pose:\n", q_sol.fkine())
```

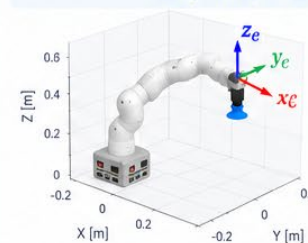
7. Result Visualization

Desired pose

Position [m]: [0.30, 0.10, 0.20]
RPY [deg]: [0, 90, 0]

IK solution

q [deg]: [12.5, -45.1, 89.6, -30.2, 41.7, 10.0]



8. Key Takeaways

- ✓ Inverse kinematics maps a desired end-effector pose to joint angles.
- ✓ There can be multiple or no solutions.
- ✓ Analytical solutions are fast but only available for specific robots.
- ✓ Numerical methods (e.g. Jacobian-based, damped least squares) are general and widely used.
- ✓ Joint limits, collisions, and task constraints must be considered.
- ✓ Validate the solution by forward kinematics and visual inspection (e.g. in RViz2 or MATLAB/Python).

Assignment Inverse Kinematics

Inverse Kinematics Assignment – Overview

Compute joint angles for a desired end-effector pose (myCobot 280)

1. Goal and Tasks

Goal

Determine joint angles $q = [\theta_1, \dots, \theta_6]^T$ that achieve a desired end-effector pose

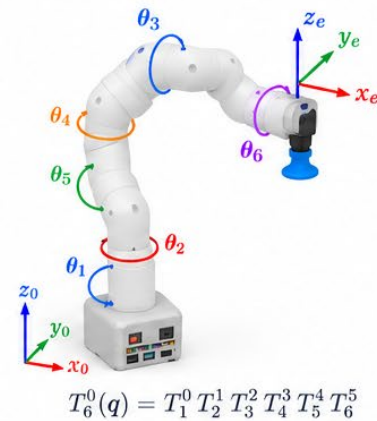
$T_6^0 = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} \in SE(3)$ for the myCobot 280 manipulator.

Tasks

- 1 Implement and test analytical IK for a simple manipulator (e.g. 2-link planar arm).
- 2 Use numerical IK for the myCobot 280 (e.g. Levenberg-Marquardt).
- 3 Validate the solution with forward kinematics.
- 4 Visualize and analyze the results in Python and in ROS 2 (RViz2).

2. Kinematic Model

Serial manipulator (myCobot 280)



$$T_6^0(q) = T_1^0 T_2^1 T_3^2 T_4^3 T_5^4 T_6^5$$

3. Inverse Kinematics Formulation

Given

Desired end-effector pose:

$$T_6^0 = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} \in SE(3)$$

- Position $p \in \mathbb{R}^3$
- Orientation $R \in SO(3)$ (e.g. as rotation matrix, RPY or quaternion)

Find joint angles $q = [\theta_1, \dots, \theta_6]^T$ such that

$$f(q) = \log \left((T_6^0(q))^{-1} T_6^0 \right) = 0$$

Pose error (6D)

$$f(q) = \begin{bmatrix} \dot{p}(q) - \dot{p}_d \\ e_R(R(q), R_d) \end{bmatrix} \in \mathbb{R}^6$$

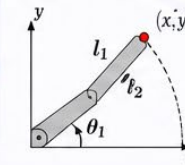
- Position error: $\dot{p}(q) - \dot{p}_d \in \mathbb{R}^3$
- Orientation error: $e_R \in \mathbb{R}^3$ (e.g. axis-angle, rotation vector or quaternion-based)

4. Solution Methods

Analytical IK (closed form)

- Possible for simple robot structures (e.g. 2R planar arm, 6R with spherical wrist).
- Geometric approach to compute joint angles from the desired pose.
- Can provide multiple solutions.

2-link planar arm (example)



Numerical IK (iterative)

- Formulate as nonlinear least-squares optimization problem.
- Methods: Newton-Raphson, Levenberg-Marquardt (LM).
- Works for general robots (e.g. myCobot 280).
- Requires initial guess and joint limits.

Minimize pose error:

$$\min_q \|f(q)\|^2$$

Update (LM):

$$\Delta q = (J^T J + \lambda I)^{-1} J^T f(q)$$
$$q_{k+1} = q_k + \Delta q$$

5. Multiple Solutions and Constraints

A desired pose can have multiple IK solutions.



Consider additional constraints:

- Joint limits: $\dot{q}_{min} \leq \dot{q} \leq \dot{q}_{max}$
- Collision avoidance
- Preferred configuration (e.g. closest to current q)
- Task-specific constraints (position, orientation)

6. Example: Numerical IK in Python

```
from roboticstoolbox import DHRobot, RevoluteDH
from spatialmath import SE3
import numpy as np

# create myCobot 280 model (example DH parameters)
robot = DHRobot(dh_params, name='myCobot280')

# desired pose (position + RPY orientation)
T_des = SE3(0.30, 0.10, 0.20) * SE3.RPY([0, 90, 0], 'deg')

# initial guess
q0 = np.zeros(6)

# solve IK using Levenberg-Marquardt
q_sol = robot.ikine_LM(T_des, q0=q0, mask=[1,1,1,1,1,1])

print('Joint angles [deg]:', q_sol.q * 180/np.pi)
print('Achieved pose:\n', q_sol.fkine())
```

Result (example)

q [deg]: [12.5 -45.1 89.6 -30.2 41.7 10.0]

Achieved pose T_6^0 :

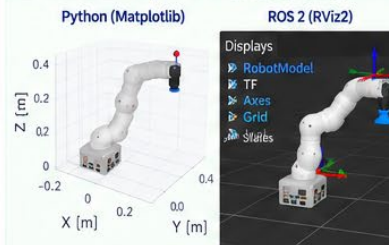
0.00	-1.00	0.00	0.30
1.00	0.00	0.00	0.10
0.00	0.00	1.00	0.20
0.00	0.00	0.00	1.00

7. Validation and Visualization

1. Validate with forward kinematics

$$T_{check} = T_6^0(q_{sol})$$
$$\dot{e}_p = \|p_{check} - p_d\| \quad \dot{e}_R = \|e_R(R_{check}, R_d)\|$$

2. Visualize in Python and ROS 2 (RViz2)



8. Key Takeaways

- ✓ Inverse kinematics maps a desired end-effector pose to joint angles.
- ✓ Solutions can be multiple, unique, or non-existent (depending on the workspace).
- ✓ Analytical solutions are possible for simple manipulators; numerical methods are general and widely used.
- ✓ Consider joint limits, collisions, and task-specific constraints.
- ✓ Validate the solution using forward kinematics.
- ✓ Use Python robotics libraries and visualize the results in ROS 2 (RViz2).

Differential Kinematics

Differential Kinematics of Robotic Manipulators

Relating joint velocities to end-effector linear and angular velocities

1. Concept and Motivation

Differential kinematics describes the instantaneous relation between joint velocities and the linear and angular velocity of the end-effector.

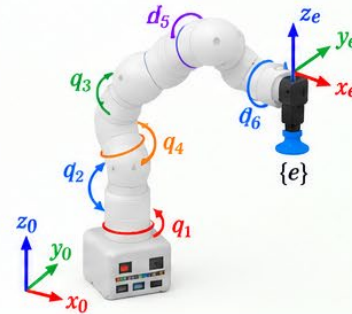
$$\begin{bmatrix} \mathbf{v} \\ \boldsymbol{\omega} \end{bmatrix} = \mathbf{J}(\mathbf{q}) \dot{\mathbf{q}}$$

- Maps joint space velocities to task space velocities.
- Based on the time derivative of the forward kinematics.
- Fundamental for velocity control, trajectory tracking and motion planning.

2. Kinematic Model

Consider an n -DOF serial manipulator with end-effector pose

$$T_e^0(\mathbf{q}) = \begin{bmatrix} \mathbf{R}(\mathbf{q}) & \mathbf{p}(\mathbf{q}) \\ \mathbf{0}^T & 1 \end{bmatrix} \in SE(3)$$



3. Jacobian Matrix

The geometric Jacobian relates joint velocities to the end-effector linear velocity $\mathbf{v}$ and angular velocity $\boldsymbol{\omega}$:

$$\begin{bmatrix} \mathbf{v} \\ \boldsymbol{\omega} \end{bmatrix} = \mathbf{J}(\mathbf{q}) \dot{\mathbf{q}}$$

For a revolute joint i :

$$\mathbf{J}_{v,i} = \mathbf{z}_{i-1} \times (\mathbf{p}_e - \mathbf{p}_{i-1})$$

$$\mathbf{J}_{\omega,i} = \mathbf{z}_{i-1}$$

For a prismatic joint i :

$$\mathbf{J}_{v,i} = \mathbf{z}_{i-1} \quad \mathbf{J}_{\omega,i} = \mathbf{0}$$

With $\mathbf{z}_{i-1}$: joint axis in base frame,
 $\mathbf{p}_{i-1}$: position of joint i ,
 $\mathbf{p}_e$: end-effector position.

4. Velocity Kinematics

Linear and angular velocity of the end-effector

$$\begin{bmatrix} \mathbf{v} \\ \boldsymbol{\omega} \end{bmatrix} = \begin{bmatrix} \dot{\mathbf{p}} \\ \dot{\boldsymbol{\theta}} \end{bmatrix} = \mathbf{J}(\mathbf{q}) \dot{\mathbf{q}}$$

- $\mathbf{v} \in \mathbb{R}^3$: linear velocity [m/s]
- $\boldsymbol{\omega} \in \mathbb{R}^3$: angular velocity [rad/s]

Time derivative of pose

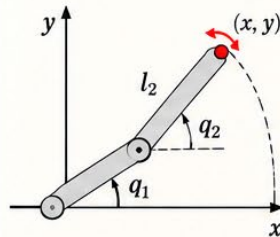
$$\dot{T} = T \begin{bmatrix} [\boldsymbol{\omega}]_{\times} & \mathbf{v} \\ \mathbf{0}^T & 0 \end{bmatrix}$$

- $[\boldsymbol{\omega}]_{\times}$: skew-symmetric matrix of $\boldsymbol{\omega}$
- Links translational and rotational motion

Jacobian properties

- $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{6 \times n}$ for a 3D manipulator
- Depends on joint configuration $\mathbf{q}$
- Maps joint velocities to end-effector twist $(\mathbf{v}, \boldsymbol{\omega})$
- Singular configurations: $\text{rank}(\mathbf{J}) < 6$
- Used for velocity control, differential motion, and redundancy resolution

5. Jacobian Example (Planar 2-Link Arm)



End-effector position:

$$x = l_1 \cos q_1 + l_2 \cos(q_1 + q_2)$$

$$y = l_1 \sin q_1 + l_2 \sin(q_1 + q_2)$$

Jacobian:

$$\mathbf{J}(\mathbf{q}) = \begin{bmatrix} -l_1 \sin q_1 - l_2 \sin(q_1 + q_2) & -l_2 \sin(q_1 + q_2) \\ l_1 \cos q_1 + l_2 \cos(q_1 + q_2) & l_2 \cos(q_1 + q_2) \end{bmatrix}$$

6. Applications



Velocity Control

Compute joint velocities for a desired end-effector velocity.



Trajectory Tracking

Use Jacobian to relate Cartesian and joint space trajectories.



Redundancy Resolution

Exploit the Jacobian's null space for secondary objectives.



Manipulability Analysis

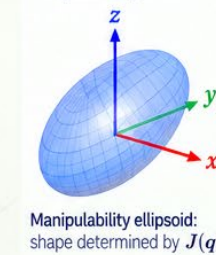
Use $\mathbf{J}(\mathbf{q})$ to analyze dexterity and singularities.

7. Visualization

Jacobian mapping



Manipulability



8. Practical Tasks in the Assignment

Derive and implement the Jacobian for the myCobot 280 (DH model)

Compute end-effector linear and angular velocities for given $\dot{\mathbf{q}}$

Implement velocity control to follow a Cartesian trajectory

Analyze singularities and manipulability

Visualize results in Python and ROS 2 (RViz2)

9. Key Takeaways

- ✓ The Jacobian links joint and Cartesian velocities: $\begin{bmatrix} \mathbf{v} \\ \boldsymbol{\omega} \end{bmatrix} = \mathbf{J}(\mathbf{q}) \dot{\mathbf{q}}$.
- ✓ It is derived from the forward kinematics and depends on the joint configuration.
- ✓ Used for velocity control, trajectory tracking, redundancy resolution and manipulability analysis.
- ✓ Singularities occur when the Jacobian loses rank, leading to reduced motion capabilities.

Resolved Rate Motion Control

Resolved-Rate Motion Control

Cartesian velocity control using the Jacobian inverse

1. Concept and Idea

Resolved-rate motion control uses the differential kinematics to compute joint velocities that realize a desired end-effector Cartesian velocity.

- Specified motion in task space (position/orientation or trajectory)
- Compute desired end-effector velocity from a tracking error
- Map to joint velocities via the Jacobian (inverse or pseudo-inverse)
- Integrate joint velocities to obtain joint motion

2. Control Law

Given a desired end-effector velocity $\mathbf{v}_d = \begin{bmatrix} \dot{\mathbf{p}}_d \\ \dot{\mathbf{q}}_d \end{bmatrix}$ and the current Jacobian $\mathbf{J}(\mathbf{q})$, the joint velocities are computed as

$$\dot{\mathbf{q}} = \mathbf{J}^{\dagger}(\mathbf{q}) \mathbf{v}_d$$

where $\mathbf{J}^{\dagger}$ is the inverse (square, non-singular) or the pseudo-inverse (redundant or singular configurations).

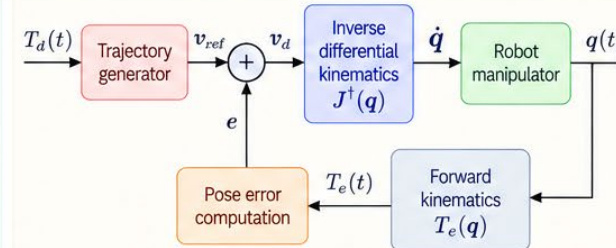
With feedback for trajectory tracking

$$\mathbf{v}_d = \mathbf{v}_{ref} + \mathbf{K}_p \mathbf{e}$$

$$\dot{\mathbf{q}} = \mathbf{J}^{\dagger}(\mathbf{q}) (\mathbf{v}_{ref} + \mathbf{K}_p \mathbf{e})$$

e.g. using pose error $\mathbf{e} = \begin{bmatrix} \mathbf{e}_\omega \\ \mathbf{e}_p \end{bmatrix}$ (orientation and position error).

3. Control Structure



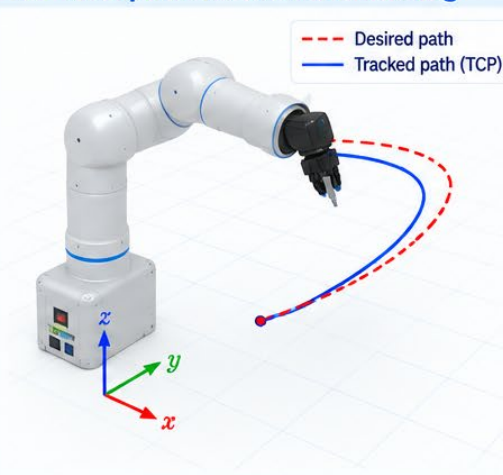
Integration of joint velocities

Joint velocities are integrated to obtain joint positions:

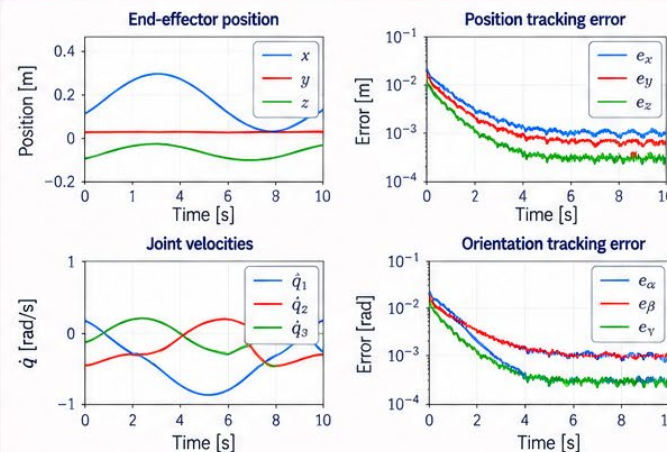
$$\mathbf{q}(t + \Delta t) = \mathbf{q}(t) + \dot{\mathbf{q}}(t) \Delta t$$

e.g. using Euler or Runge-Kutta integration.

4. Example: Cartesian Path Tracking



5. Results



6. Key Points and Variations

- Uses the geometric Jacobian to map Cartesian velocities to joint velocities.
- Suitable for real-time Cartesian motion control.
- Requires Jacobian inverse or pseudo-inverse (handle redundancy and singularities).
- Tracking based on pose error feedback.
- Common for path following, visual servoing, and compliant motion.

Variations

- Damped least-squares inverse for singularity robustness

$$\mathbf{J}^{\dagger} = \mathbf{J}^T (\mathbf{J} \mathbf{J}^T + \lambda^2 \mathbf{I})^{-1}$$

- Null-space motion for secondary objectives

$$\dot{\mathbf{q}} = \mathbf{J}^{\dagger} \mathbf{v}_d + (\mathbf{I} - \mathbf{J}^{\dagger} \mathbf{J}) \dot{\mathbf{q}}_0$$

- Task-space weighting (position vs. orientation)

Assignment Differential Kinematics

Differential Kinematics Assignment – Overview

Relating joint velocities to end-effector linear and angular velocities

1. Goal and Learning Objectives

Goal

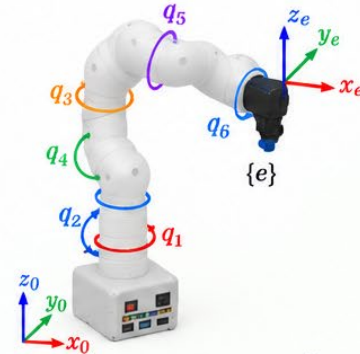
Understand and apply differential kinematics to relate joint velocities $\dot{q}$ to the end-effector linear and angular velocities $\nu_e = [\omega_e^T \dot{p}_e^T]^T$ for the myCobot 280 manipulator.

After completing the assignment you will:

- 1 Derive and interpret the geometric Jacobian.
- 2 Compute joint velocities for a desired end-effector velocity.
- 3 Analyze singularities, rank and null space.
- 4 Implement resolved-rate motion control.
- 5 Integrate velocity commands to generate joint trajectories.
- 6 Validate and visualize results in Python and (optionally) in ROS 2 (RViz2).

2. Kinematic Model

Serial manipulator (myCobot 280)



Joint configuration: $q = [q_1 \dots q_6]^T$

End-effector pose: $T_e^0(q) = \begin{bmatrix} R(q) & p(q) \\ 0^T & 1 \end{bmatrix}$

3. Geometric Jacobian

The geometric Jacobian $J(q) \in \mathbb{R}^{6 \times n}$ relates joint velocities to the end-effector linear and angular velocity:

$$\begin{bmatrix} \omega_e \\ \dot{p}_e \end{bmatrix} = J(q) \dot{q}$$

Revolute joint i :

$$J_{\omega, i} = z_{i-1}$$

$$J_{v, i} = z_{i-1} \times (p_e - p_{i-1})$$

Prismatic joint i :

$$J_{\omega, i} = 0$$

$$J_{v, i} = z_{i-1}$$

z_{i-1} : joint axis, p_{i-1} : position of joint i , p_e : end-effector position.

4. Applications and Tasks



Velocity control

Compute joint velocities for a desired end-effector velocity ν_d .



Trajectory tracking

Use Jacobian to follow Cartesian trajectories (resolved-rate control).



Redundancy resolution

Exploit null space for secondary objectives (e.g., joint limit avoidance).



Singularity analysis

Analyze rank, singular configurations and their implications.



Frame transformations

Express Jacobian in different reference frames (e.g., base, end-effector, camera).

5. Velocity-Level Control (Resolved-Rate)

Compute joint velocities from a desired end-effector velocity using the (pseudo-)inverse of the Jacobian.

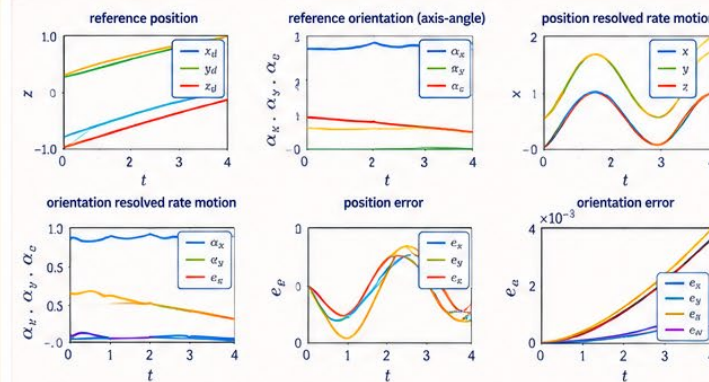
$$\dot{q} = J(q)^+ \nu_d$$

- Use pseudo-inverse J^+ for non-square or redundant robots.
- Optionally add secondary tasks in the null space:

$$\dot{q} = J^+ \nu_d + (I - J^+ J) \dot{q}_0$$

- Integrate joint velocities to obtain joint trajectories (e.g., Euler or Runge-Kutta methods).

6. Example Results (Task-Space Trajectory)



7. Practical Tasks in the Assignment

- 1 Compute the Jacobian for the myCobot 280 using the Robotics Toolbox for Python.
- 2 Generate joint velocities for a given Cartesian velocity command.
- 3 Implement resolved-rate control to follow a Cartesian trajectory.
- 4 Analyze Jacobian rank and singularities along a trajectory.
- 5 Integrate joint velocities to obtain joint trajectories (Euler / Runge-Kutta).
- 6 Visualize and validate results in Python and in ROS 2 (RViz2).

Assignment TCP Calibration

TCP Calibration Assignment – Overview

Estimating the Tool Center Point (TCP) for accurate Cartesian robot motion

1. Motivation and Goal

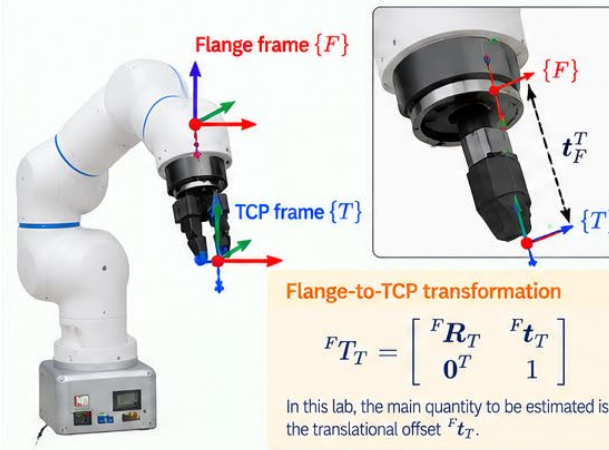
In practice, a tool is mounted to the robot flange. The Tool Center Point (TCP) is the point on the tool that is relevant for the task (e.g., gripper center, pen tip, probe).

This assignment shows how to estimate the geometric offset between the robot flange and the actual TCP on the myCobot 280 and how to use the calibrated TCP for accurate Cartesian motions.

Learning Objectives

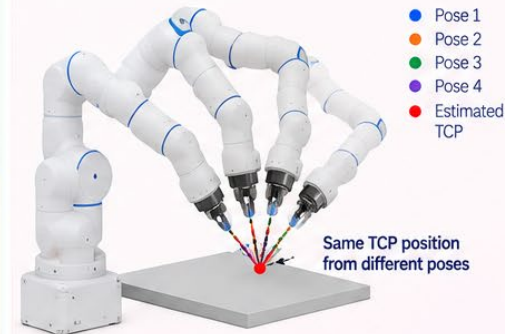
- 1 Understand the role of the TCP in robot programming and execution.
- 2 Collect measurements on the real robot.
- 3 Estimate a flange-to-TCP offset.
- 4 Validate the calibration result.
- 5 Apply the calibrated TCP in a robot workflow.

2. Flange and TCP Concept



3. Calibration Idea

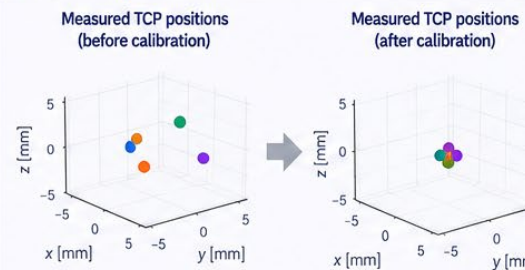
Move the TCP to the same fixed point in the workspace from multiple different robot poses. If the TCP is correct, the measured points coincide. The flange poses are used to estimate a single flange-to-TCP offset.



4. Calibration Workflow

- 1 **Data Acquisition**
 - Move the TCP to a fixed point from several different poses.
 - Record joint angles and flange poses.
- 2 **Offset Estimation**
 - Estimate the flange-to-TCP offset ${}^F t_T$ from the collected poses.
 - Use a least-squares formulation.
- 3 **Validation**
 - Compute residual / Cartesian position error.
 - Analyze consistency of the results.
- 4 **Use Calibrated TCP**
 - Apply the estimated TCP in a practical robot motion.
 - Verify improved positioning accuracy.

5. Example Results



Error Analysis

- Cartesian position error of repeated TCP measurements
- Residual of the estimated offset
- Visual inspection of the measured points
- Validation by executing test motions with the calibrated TCP

6. Practical Setup (myCobot 280)



7. Key Takeaways

- ✓ The TCP is the physically relevant tool point for Cartesian motions.
- ✓ The flange-to-TCP offset is a rigid geometric parameter of the tool.
- ✓ Use multiple measurement poses to estimate one consistent offset.
- ✓ Validate the result with suitable error measures.
- ✓ A correct TCP significantly improves Cartesian positioning accuracy and repeatability.

Trajectory Planning

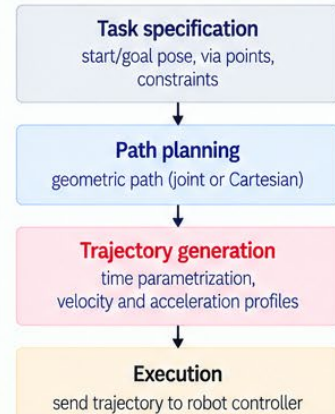
Robot Arm Trajectory Planning

From task specification to smooth, feasible joint and Cartesian trajectories

1. Goal and Motivation

- Plan smooth and feasible joint and Cartesian trajectories for a robot arm to move from a start to a goal pose.
- Satisfy kinematic and dynamic constraints (joint limits, velocity, acceleration).
- Generate collision-free and smooth motions for pick-and-place and manipulation tasks.
- Use trajectories for simulation and real robot execution (e.g., the myCobot 280).

2. Trajectory Planning Pipeline



3. Path vs. Trajectory

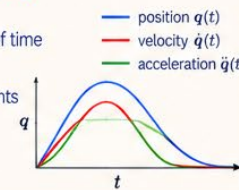
Path (geometric)

- Sequence of positions in joint or Cartesian space
- Independent of time



Trajectory (time-parameterized)

- Positions, velocities and accelerations as functions of time
- Satisfies kinematic limits and smoothness requirements



4. Trajectory Generation Methods

Joint-space interpolation

- Plan each joint trajectory independently
- E.g., polynomial, trapezoidal or S-curve profiles

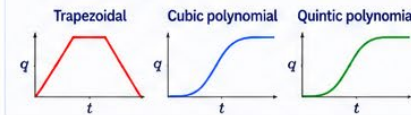
$$q(t) = [q_1(t) \ \dots \ q_n(t)]^T$$

Cartesian trajectory planning

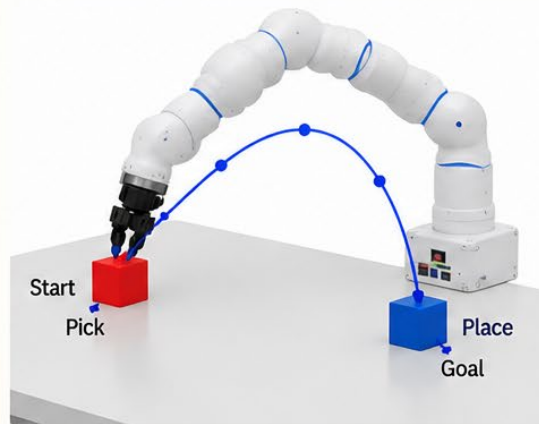
- Plan end-effector pose in Cartesian space
- Convert to joint space using inverse kinematics

$$T_e(t) = \begin{bmatrix} R(t) & p(t) \\ 0^T & 1 \end{bmatrix} \rightarrow q(t)$$

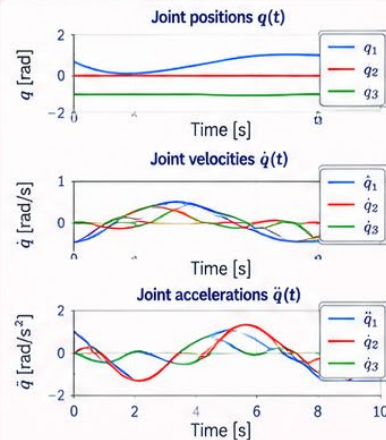
Common time profiles



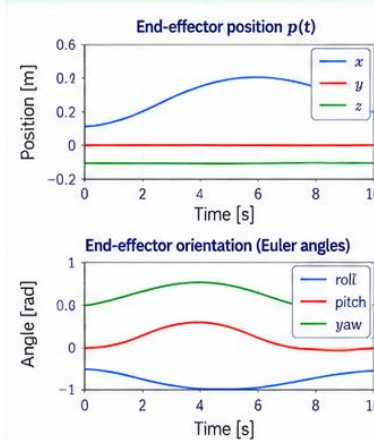
5. Example: Pick-and-Place Trajectory



6. Joint Trajectories



7. Cartesian Trajectory



8. Key Considerations

- 1 Satisfy joint limits (position, velocity, acceleration).
- 2 Ensure smooth trajectories (e.g., cubic or quintic polynomials).
- 3 Consider collision avoidance and workspace constraints.
- 4 Use IK for Cartesian paths and handle multiple solutions.
- 5 Synchronize motion between joints.
- 6 Validate the trajectory in simulation before real robot execution.

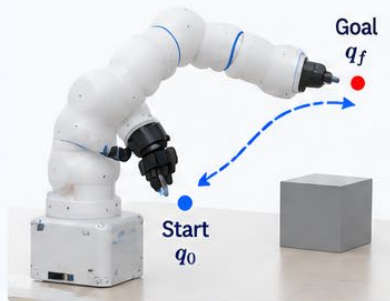
Assignment Trajectory Planning

Trajectory Planning Assignment – Overview

Smooth point-to-point robot motion with velocity and acceleration constraints

1. Goal and Motivation

Plan smooth and feasible joint trajectories for point-to-point motion of the myCobot 280 under velocity and acceleration constraints.

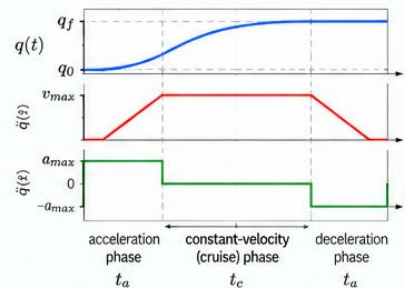


Learning Objectives

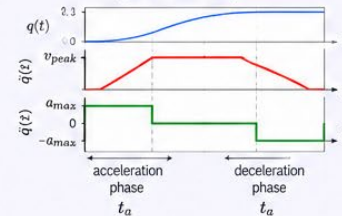
- Understand trapezoidal and triangular velocity profiles.
- Compute phase durations from limits.
- Generate feasible joint trajectories.
- Use the trajectories in the myCobot workflow.

2. Trajectory Types

Trapezoidal velocity profile
(longer motion)



Triangular velocity profile
(shorter motion)



3. Key Formulas

Joint motion from q_0 to q_f

$$\Delta q = q_f - q_0$$

Trapezoidal profile ($|\Delta q| > \Delta q_{accdec}$)

$$\text{Acceleration time } t_a = \frac{v_{max}}{a_{max}}$$

$$\text{Distance for acceleration and deceleration } \Delta q_{accdec} = \frac{v_{max}^2}{a_{max}}$$

$$\text{Cruise time } t_c = \frac{|\Delta q| - v_{max}^2/a_{max}}{v_{max}}$$

$$\text{Total motion time } t_f = 2t_a + t_c$$

Triangular profile ($|\Delta q| \leq \Delta q_{accdec}$)

$$\text{Acceleration time } t_a = \sqrt{\frac{|\Delta q|}{a_{max}}}$$

$$\text{Peak velocity } v_{peak} = a_{max} t_a$$

$$\text{Total motion time } t_f = 2t_a$$

The sign of Δq determines the direction of motion. Phase durations are computed from $|\Delta q|$.

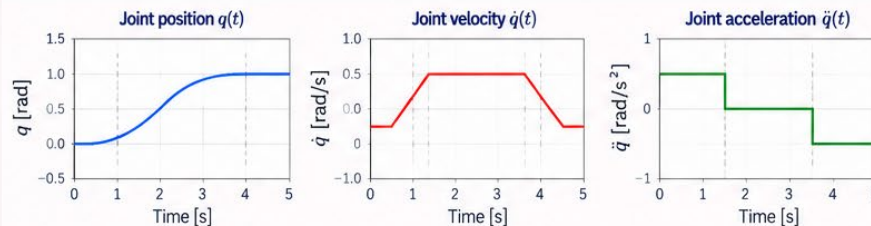
4. Workflow for the Assignment

- 1 Define start and goal configuration q_0, q_f (joint space).
- 2 Specify velocity and acceleration limits v_{max}, a_{max} .
- 3 Compute trajectory parameters t_a, t_c (if needed) and t_f .
- 4 Generate joint trajectories $q(t), \dot{q}(t), \ddot{q}(t)$.
- 5 Use trajectories in simulation and on the myCobot 280 for point-to-point motion.

Constraints and Requirements

- Satisfy joint velocity and acceleration limits.
- Generate smooth position and velocity profiles (continuous $q(t)$ and $\dot{q}(t)$).
- Handle trapezoidal and triangular cases.
- Validate trajectories before execution.
- Use the provided Python implementation (`_build_ee_profile()`) in the myCobot codebase.

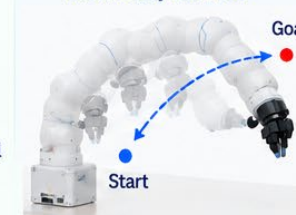
5. Example Trajectory for One Joint



6. Practical Considerations

- Respect joint limits of the myCobot 280.
- Choose appropriate v_{max} and a_{max} for smooth and safe motion.
- Ensure synchronization of multiple joints.
- Check for collisions and workspace constraints.
- Use the generated trajectories in the experimental workflow with Python and Jupyter notebooks.

Result on myCobot 280



Vision Based Object Grasping

Vision-Based Object Grasping with a Robot Arm

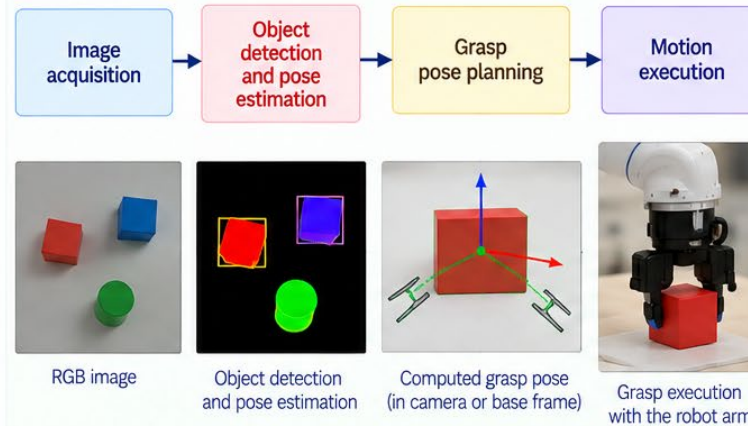
From camera input to grasp execution

1. Goal and Motivation

- Enable the robot to autonomously detect, localize and grasp objects using a camera.
- Combine computer vision, geometric reasoning and robot control in a complete pipeline.
- Apply the method to the myCobot 280 with a parallel gripper.

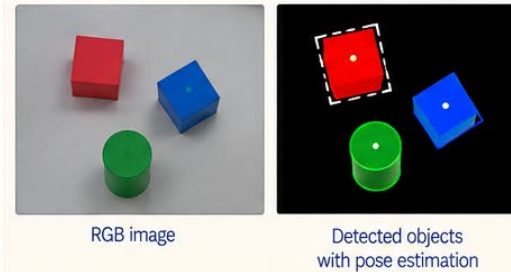


2. Overall Pipeline



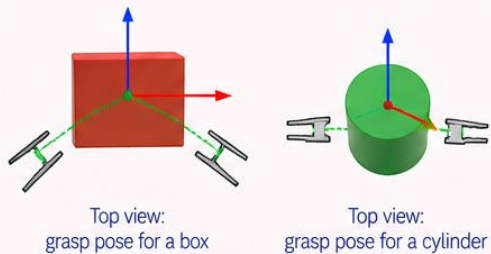
3. Object Detection and Pose Estimation

- Detect objects in the image (e.g., by color segmentation, deep learning, or classical methods).
- Estimate the object pose: position (x, y, z) and orientation in the robot base frame.
- Use camera calibration and hand-eye calibration to transform the object pose from the camera frame to the robot base frame.



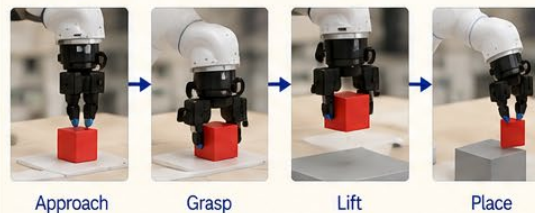
4. Grasp Planning

- Define a suitable grasp for the detected object.
- Compute the grasp pose (position and orientation).
- Consider the object shape, size and approach direction.
- Transform the grasp pose to the robot base frame.

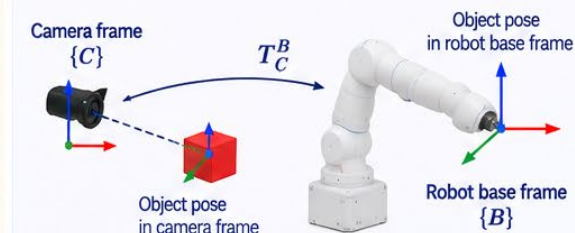


5. Robot Motion Execution

- Plan a collision-free approach to the grasp pose.
- Move the robot to a pre-grasp position.
- Approach along the grasp direction and close the gripper.
- Lift the object and optionally move it to a target location.
- Use Cartesian or joint-space motion planning (e.g., Moveit 2, or resolved-rate control).



6. Coordinate Frames and Transformations



Key transformations

- Camera calibration: map image pixels to 3D points in camera frame.
- Hand-eye calibration: transform poses from camera frame $\{C\}$ to robot base frame $\{B\}$.
- Use homogeneous transformations: $T_O^B = T_C^B T_O^C$.

Position Based Visual Servoing

Position-Based Visual Servoing and Pose Estimation

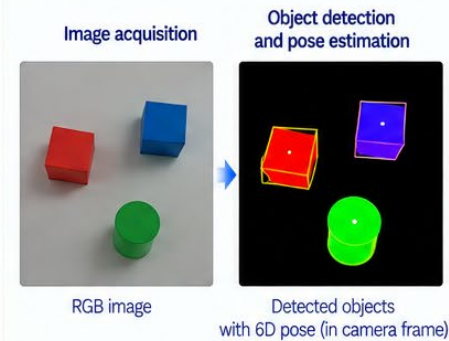
Regulating the robot end-effector to an object using a fixed camera

1. Task and Goal

- Detect an object with a fixed camera.
- Estimate the 3D pose of the object in the camera frame.
- Transform the object pose to the robot base frame.
- Use position-based visual servoing to move the robot end-effector to a desired grasp pose (above the object) and grasp it.

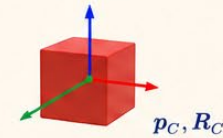


2. Pose Estimation from a Fixed Camera Pose



Object pose in camera frame $\{C\}$

- Position: $p_C = [x, y, z]^T$
- Orientation: $R_C \in SO(3)$



Transformation to robot base frame $\{B\}$

$$p_B = T_C^B p_C \quad T_C^B = \begin{bmatrix} R_C^B & t_C^B \\ 0^T & 1 \end{bmatrix}$$
$$R_B = R_C^B R_C$$

Known and fixed from camera calibration (hand-eye calibration).

3. Position-Based Visual Servoing

Regulate the position of the robot end-effector to the desired target position (e.g., pre-grasp or grasp pose) using a position error in Cartesian space.

Position error

$$e_p = p_d - p_e$$

- p_d : desired end-effector position (target grasp pose)
- p_e : current end-effector position (from forward kinematics)

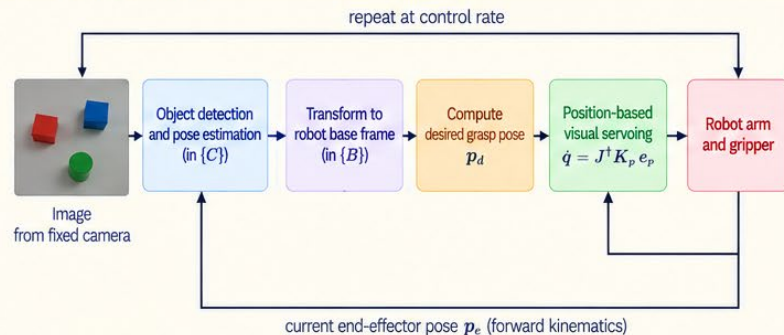
Resolved-rate control (position)

$$v_d = K_p e_p$$

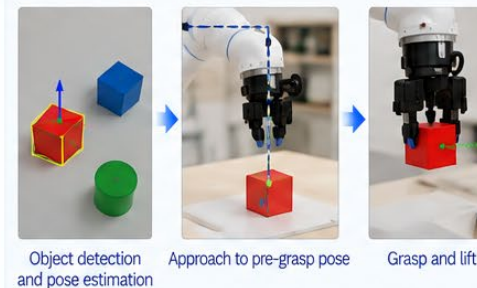
$$\dot{q} = J^\dagger(q) v_d$$

- $J^\dagger$: Jacobian pseudo-inverse
- K_p : positive definite gain matrix

4. Control Loop



5. Experimental Results



6. Key Takeaways

- 1 3D object pose can be estimated from a single fixed camera using calibration and geometric vision methods.
- 2 Transform object pose to the robot base frame using a fixed hand-eye transformation.
- 3 Position-based visual servoing uses a Cartesian position error and the Jacobian to compute joint velocities.
- 4 Enables robust and intuitive grasping of objects in a fixed workspace.

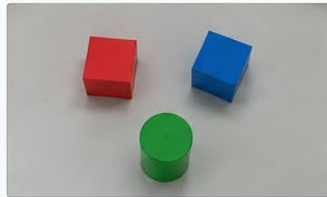
Color Based Object Segmentation

Color-Based Segmentation of Objects in Images

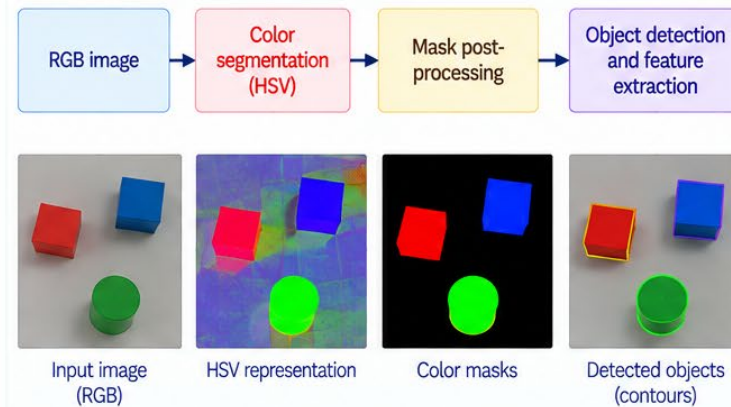
Detect and segment colored objects for robotic manipulation

1. Goal and Motivation

- Detect and segment objects in a camera image based on their color.
- Identify the object pixels, compute object contours, and extract relevant features such as position, size and shape.
- Use the detected objects for subsequent tasks such as grasping with a robot arm.

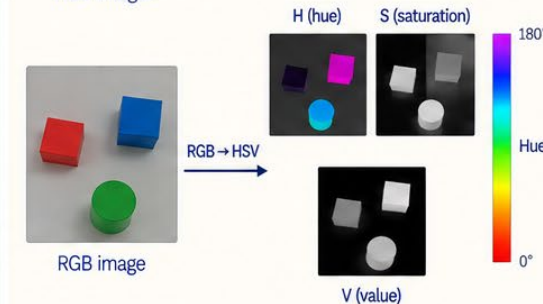


2. Processing Pipeline



3. Color Segmentation in HSV Space

- Convert the RGB image to HSV color space, which separates color (hue) from intensity.
- Define a threshold range for each color.
- Create a binary mask for each color by thresholding the HSV image.



4. Thresholding and Mask Creation

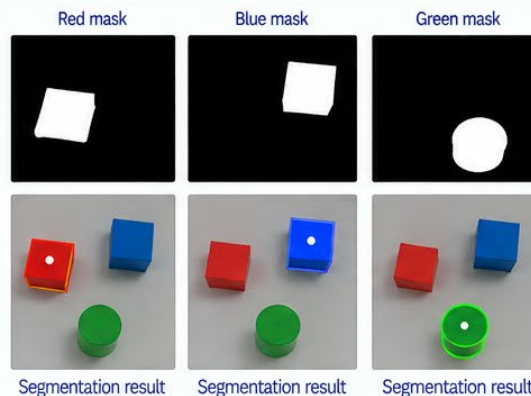
Binary mask for a color (e.g., red):

$$M(x, y) = \begin{cases} 1 & \text{if } h_{min} \leq H(x, y) \leq h_{max} \\ & s_{min} \leq S(x, y) \leq s_{max} \\ & v_{min} \leq V(x, y) \leq v_{max} \\ 0 & \text{otherwise} \end{cases}$$

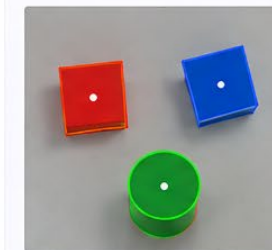
Typical steps:

- 1 Convert RGB to HSV.
- 2 Choose lower and upper HSV thresholds for each color.
- 3 Create binary mask using `inRange()`.
- 4 Apply morphological operations (e.g., opening/closing) to remove noise.

5. Example Results



6. Object Detection and Feature Extraction



Extracted features per object:

- Pixel position (center): (u, v)
- Bounding box: width, height
- Area (number of pixels)
- Shape information (e.g., aspect ratio, circularity)
- Color label (e.g., red, blue, green)

Use for robot manipulation:

- Determine object position in the camera image.
- Transform to robot base frame (using camera calibration).
- Select a grasp pose based on object position and shape.
- Execute grasp with the robot arm.

AruCo Markers and Camera to Robot Frame Transform Estimation

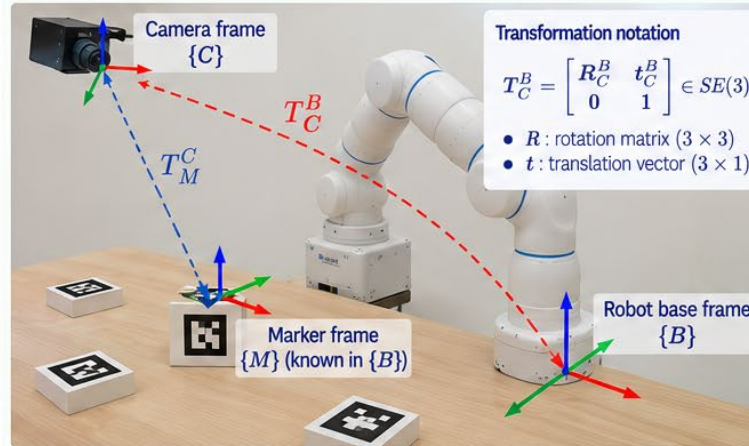
Use known marker poses to estimate the camera-to-robot transformation

1. Goal and Motivation

- Determine the rigid transformation between the camera frame $\{C\}$ and the robot base frame $\{B\}$.
- Use ArUco markers with known poses in the robot base frame.
- Detect the markers in the camera image and estimate their poses in the camera frame.
- Compute the camera-to-robot transform and use it for vision-based manipulation tasks (e.g., grasping, visual servoing).

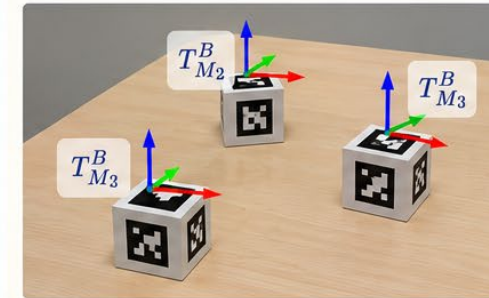


2. Coordinate Frames and Transformations



3. Known Marker Poses in Robot Frame

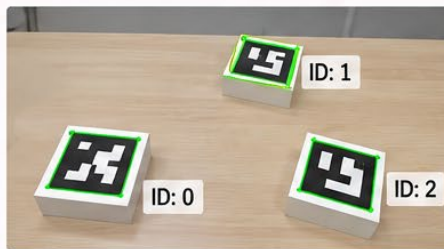
- Place several ArUco markers at known poses in the robot base frame $\{B\}$.
- For each marker i , the pose $T_{M_i}^B = \begin{bmatrix} R_{M_i}^B & t_{M_i}^B \\ 0 & 1 \end{bmatrix}$ is known from measurement or the setup design.



4. Marker Detection and Pose Estimation

- Detect ArUco markers in the camera image (e.g., using OpenCV: `cv2.aruco`).
- Estimate the pose of each detected marker i in the camera frame $\{C\}$:

$$T_{M_i}^C = \begin{bmatrix} R_{M_i}^C & t_{M_i}^C \\ 0 & 1 \end{bmatrix}$$



Detected markers in camera image

5. Estimation of T_C^B

- For each detected marker i , the following relation holds:

$$T_C^B \cdot T_{M_i}^C = T_{M_i}^B$$

- Given multiple markers, estimate T_C^B by minimizing the pose error (e.g., using least squares):

$$\min_{T_C^B} \sum_i \left\| \log \left((T_C^B T_{M_i}^C)^{-1} T_{M_i}^B \right) \right\|^2$$

Result

Estimated camera-to-robot transform:

$$T_C^B = \begin{bmatrix} R_C^B & t_C^B \\ 0 & 1 \end{bmatrix}$$

- Use multiple markers for a robust estimate.
- Validate the result by checking the reprojection or pose error.

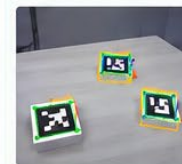
6. Validation

- Transform the detected marker poses to the robot frame using the estimated T_C^B :

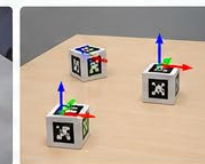
$$\hat{T}_{M_i}^B = T_C^B \cdot T_{M_i}^C$$

- Compare with the known marker poses $T_{M_i}^B$.

Marker ID	Position error [mm]	Orientation error [°]
0	1.2	0.5
1	1.8	0.7
2	1.5	0.6
Mean	1.5	0.6



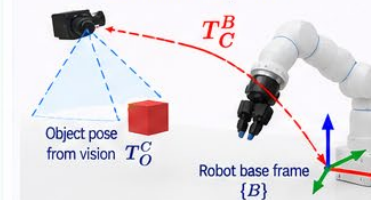
Reprojection result



Estimated marker poses in robot frame

7. Use in Robot Applications

- Use the estimated camera-to-robot transform to convert object poses from the camera frame to the robot base frame.
- Enable tasks such as
 - object localization and grasping
 - pick-and-place
 - position-based visual servoing
 - autonomous manipulation



Combines vision and robot kinematics for real-world manipulation

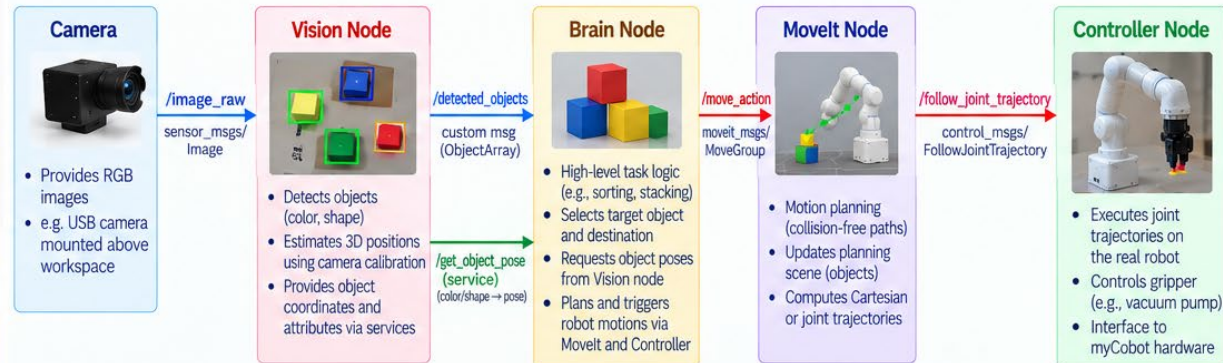
Vision Based Pick & Place Architecture

ROS 2 Nodes in the Pick & Place Setup

From camera images to autonomous pick-and-place execution

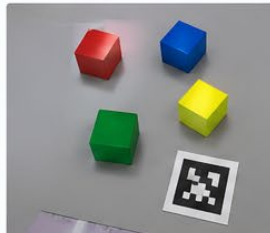


ROS 2 Architecture and Data Flow



1. Camera Node

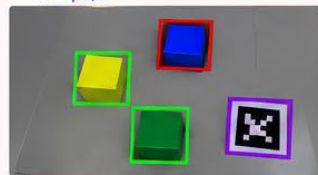
- Captures RGB images of the workspace.
- Publishes images on `/image_raw` (`sensor_msgs/Image`).



RGB image from camera

2. Vision Node

- Segments objects using HSV color thresholds.
- Detects objects and estimates their image position and size.
- Transforms image coordinates to robot base frame using camera calibration.
- Provides object information via a service (position, color, shape).



Detected objects (color and shape)

3. Brain Node

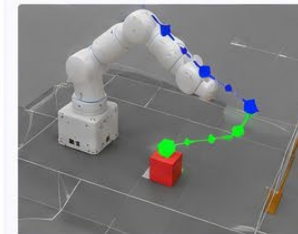
- Implements high-level pick-and-place logic (e.g., sorting, stacking).
- Queries the Vision node for object poses.
- Adds objects to the MoveIt planning scene.
- Triggers pick and place sequences (approach, grasp, lift, place).

Example tasks:

- Sort cubes by color
- Stack cubes
- Pick object selected by user (color and shape)

4. MoveIt Node

- Plans collision-free motions for the robot arm.
- Provides planning services and actions (e.g., `/move_action`).
- Maintains the planning scene with detected objects.



Planned motion in MoveIt

5. Controller Node

- Executes the planned joint trajectories on the myCobot 280.
- Controls the end effector (e.g., vacuum pump for grasping).
- Receives trajectories via `/follow_joint_trajectory`.

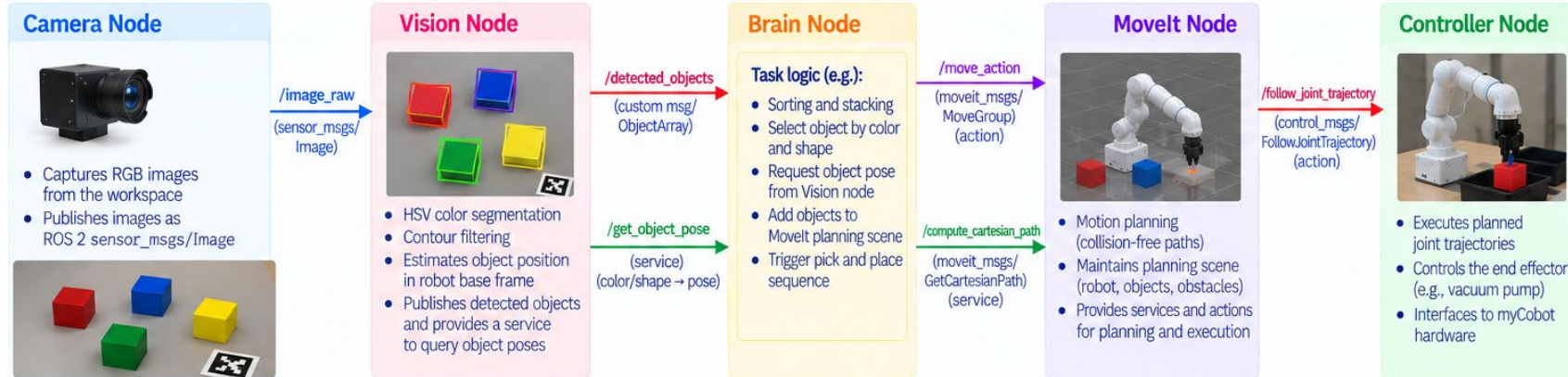


Execution on the real robot

Vision Based Pick & Place Topics & Messages

ROS 2 Topics, Services and Actions in the Pick & Place Assignment

Overview of message types and data flow between the nodes



1. Topic Messages (data stream)

Topics are used to continuously publish data.

`/image_raw` (`sensor_msgs/Image`)



- RGB image from camera
- Used by the Vision node
- Standard ROS 2 message

`/detected_objects` (custom `msg/ObjectArray`)

ObjectArray

```
objects:
- color: "red"
  shape: "cube"
  position: [0.23, -0.15, 0.02]
  size: [0.05, 0.05, 0.05]
```



- List of detected objects (color, shape, position, size)
- Published by the Vision node
- Used by the Brain node

2. Service Messages (request / response)

Services are used for request/response interactions (e.g., get object pose).

`/get_object_pose` (custom `srv/GetObjectPose`)

Request	Response
string color string shape	geometry_msgs/Pose pose bool success

- Request the pose of an object with specified color and shape
- Provided by the Vision node
- Used by the Brain node

`/compute_cartesian_path`
(`moveit_msgs/srv/GetCartesianPath`)

Request	Response
geometry_msgs/Pose[] waypoints float64 eef_step float64 jump_threshold	moveit_msgs/RobotTrajectory trajectory float64 fraction

- Compute a Cartesian path for a set of waypoints
- Provided by the MoveIt node
- Used by the Brain node (optional)

3. Action Messages (goal / feedback / result)

Actions are used for long-running tasks, such as motion execution.

`/move_action` (`moveit_msgs/action/MoveGroup`)

Goal	Feedback	Result
moveit_msgs/MotionPlanRequest request	moveit_msgs/MoveItFeedback feedback	moveit_msgs/MoveItErrorCodes error_code moveit_msgs/RobotTrajectory trajectory

- Execute a planned motion (joint or Cartesian)
- Provided by the MoveIt node
- Used by the Brain node

`/follow_joint_trajectory`
(`control_msgs/action/FollowJointTrajectory`)

Goal	Feedback	Result
trajectory_msgs/JointTrajectory trajectory	control_msgs/FollowJointTrajectory_Feedback (actual positions)	control_msgs/FollowJointTrajectory_Result (error code)

- Execute joint trajectory on the real robot
- Provided by the controller node
- Called by MoveIt (or directly if needed)

4. Other Important Messages

Additional standard messages used in the assignment.

`/collision_object`
(`moveit_msgs/msg/CollisionObject`)

- Add / remove objects in the planning scene
- Publish from Brain node to MoveIt
- Used to represent cubes, containers, table, etc.



`/attached_collision_object`
(`moveit_msgs/msg/AttachedCollisionObject`)

- Attach / detach an object to the robot
- Publish from Brain node to MoveIt
- Used after grasping an object

`JointTrajectory`
(`trajectory_msgs/msg/JointTrajectory`)

- Sequence of joint positions, velocities, etc.
- Used in `/follow_joint_trajectory` action
- Contains `joint_names`, `points` (positions, velocities, time_from_start)

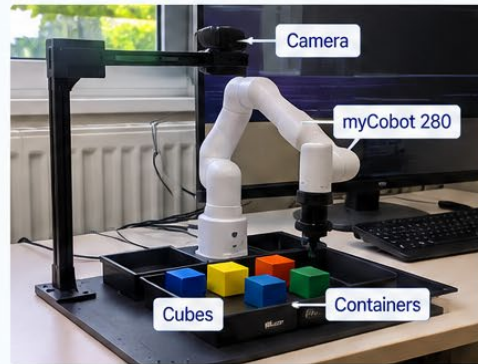
Assignment Explore Existing Pick & Place System

Assignment: Explore the Existing Pick-and-Place System

Run the provided system, use the GUI, and investigate the complete pick-and-place workflow

1. System Overview

The existing system uses a camera, vision node, Brain node, MoveIt and a controller to perform pick-and-place tasks with the myCobot 280.



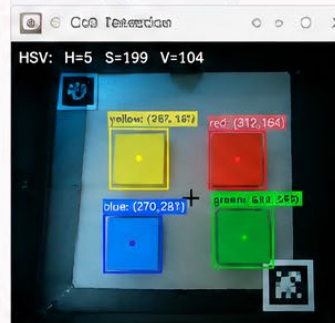
2. Existing GUI and Control Menu

Run the provided ROS 2 nodes and use the text-based control menu in **brain.py** to operate the system.

Control Menu (terminal output)

```
=== Pick and Place Control ===  
1 - Stacking mode  
2 - Sorting mode  
3 - Move to home  
4 - Open gripper (pump off)  
5 - Close gripper (pump on)  
6 - Quit  
  
Select mode: |
```

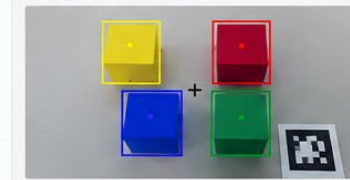
Camera display (vision node)



3. Run and Prepare the Setup

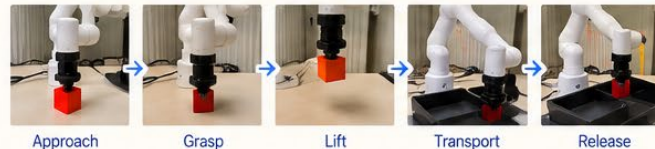
- 1 Inspect the robot workspace, camera view, cubes, containers, and vacuum pump.
- 2 Start the provided ROS 2 nodes according to the lab instructions.
- 3 Check that the vision display detects the cubes before starting a robot motion.
- 4 Perform the first motions under supervision and keep the workspace clear.

Example: detected cubes



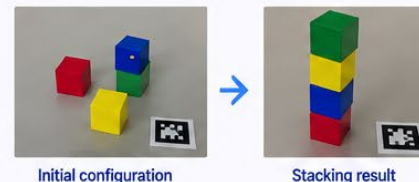
4. Try the Sorting Mode (Menu 2)

- 1 Select **mode 2** in the control menu.
- 2 Choose a detected cube by color and select a destination container.
- 3 Repeat the procedure with at least two cubes and different containers.
- 4 Observe the approach, grasp, lift, transport, release, and return-to-home motions.



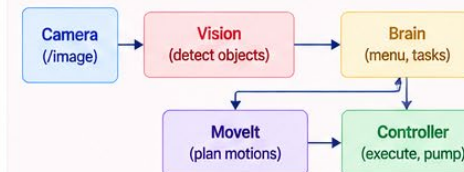
5. Try the Stacking Mode (Menu 1)

- 1 Place the cubes in suitable starting positions and select mode 1.
- 2 Observe the order in which cubes are picked and stacked.
- 3 Note what happens when a required cube is not detected or a motion cannot be completed.



6. Trace the Information Flow

- 1 Identify which node supplies the camera image.
- 2 Which node determines the cube coordinates?
- 3 How does the **Brain** node request these coordinates?
- 4 Which interfaces are used for motion planning, trajectory execution, and pump control?



7. Deliverable

Submit a short report with your observations and a diagram or ordered list showing the data flow from camera image to completed pick-and-place motion. Describe at least two limitations or unexpected behaviors (e.g., missed detections, inaccurate grasp positions, failed trajectories, or restrictions of the current menu).

Extension Tasks Pick & Place

Pick-and-Place Assignment: Overview of Extension Tasks

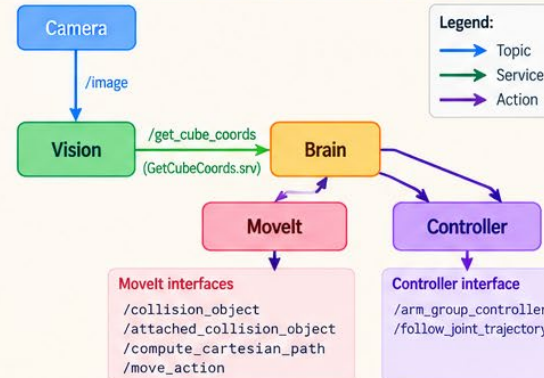
Improve the vision-guided pick-and-place system for the myCobot 280

1. Existing System



- Overhead camera captures workspace image (/image).
- Vision node detects colored cubes and provides coordinates (/get_cube_coords).
- Brain node handles user interface, calls vision service, and controls pick-and-place.
- MoveIt plans motions, Controller executes, and vacuum pump picks objects.

2. ROS 2 Architecture (Current System)



3. Extension Tasks (Choose Two)

Implement and evaluate two of the following five extension tasks. The remaining tasks are optional.

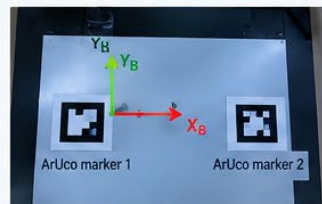
- 1 Camera calibration using ArUco markers
- 2 Robust HSV masking
- 3 Interactive calibration of HSV thresholds
- 4 Object shape detection (cubes vs. cylinders)
- 5 Extension of the object interface (color + shape)

Deliverables: Document and evaluate your results with experiments, comparison to the original system, and a diagram or ordered list showing the data flow from camera image to completed pick-and-place motion.

1 Camera Calibration (ArUco Markers)

Goal

Replace the fixed pixel-to-robot mapping with a calibrated camera model and extrinsics using two ArUco markers.



Key steps

- Obtain camera intrinsics K and distortion coefficients d .
- Use two markers to estimate the camera-to-robot transform (`cv2.solvePnP`).
- Validate with reprojection error and independent workspace points.

2 Robust HSV Masking

Goal

Improve color segmentation so that cube detection remains reliable under different lighting conditions and does not report disappeared objects.



Key steps

- Apply mask cleanup (opening/closing).
- Remove small isolated regions and fill gaps.
- Use size, aspect ratio, and area checks.
- Prevent stale detections (reset detections each frame).

3 Interactive HSV Threshold Calibration

Goal

Develop a tool that estimates suitable HSV thresholds from an object selected in an image.



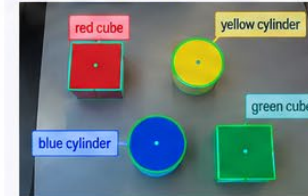
Key steps

- Let the user mark a polygon around the object (`cv2.fillPoly`).
- Extract HSV pixels inside the region.
- Compute median and standard deviation for H, S, V.
- Determine thresholds (handle hue wrap-around near 0/179).
- Apply thresholds to full image and display the resulting mask.

4 Object Shape Detection

Goal

Detect both the color and the shape of objects (cubes and cylinders). Cubes appear approximately square, cylinders approximately circular (top view).



Key steps

- Segment by color (HSV masks).
- Extract contour features: area, perimeter, min-area bounding rectangle, circularity ($C = \frac{4\pi A}{P^2}$).
- Classify as cube or cylinder using thresholds.
- Store and display multiple detections with color, shape, center, and orientation.

5 Extension of Object Interface

Goal

Extend the vision service and Brain node so that objects can be selected by both color and shape, represented correctly in MoveIt, and picked via the control menu.

Extended service (example)

```
GetObject.srv
string color      # e.g., "red"
string shape      # e.g., "cube" | "cylinder"
---
bool found
int32 id          # unique object ID
geometry_msgs/Pose pose
# optionally: list of all matches
```

Key steps

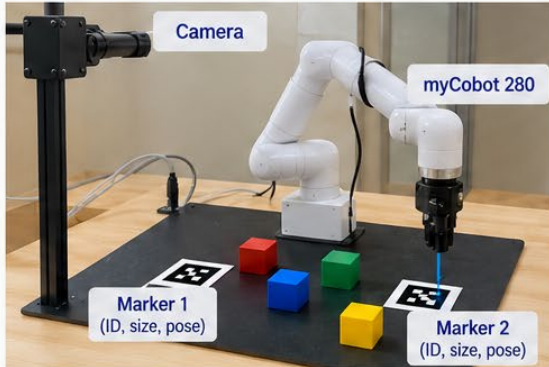
- Adapt service client: `get_object(color, shape)`.
- Represent objects in MoveIt: cube → `SolidPrimitive.BOX`, cylinder → `SolidPrimitive.CYLINDER`.
- Use unique object IDs for add/attach/detach.
- Extend terminal menu to select color and shape and handle multiple matches.

Extension Camera Calibration Task

Calibration Task: Camera-to-Robot Transformation

Replace the fixed mapping with a calibrated transformation using two ArUco markers

1. Setup and Known Quantities



- Two ArUco markers with known poses in the robot base frame B (ID, side length, position, orientation).
- Camera height h above the marker plane is known.
- Define the marker plane as $z_B = 0$ (or use its actual height).

4. Estimate Camera Extrinsics

Detect both markers in an image and use their eight corner correspondences, K , and d to estimate the robot-to-camera transformation with `cv2.solvePnP`:

$$P_C = R_{CB}P_B + t_{CB}$$

Invert to obtain the camera-to-robot transform:

$$R_{BC} = R_{CB}^T, \quad t_{BC} = -R_{CB}^T t_{CB}$$

- Check that the estimated camera height $t_{BC,z}$ agrees with the measured height h .
- Choose a pose consistent with the physical camera placement.
- Define the behavior if one of the markers is missing (e.g., fall back, do not update, or use the previous estimate).

2. Camera Model and Intrinsics

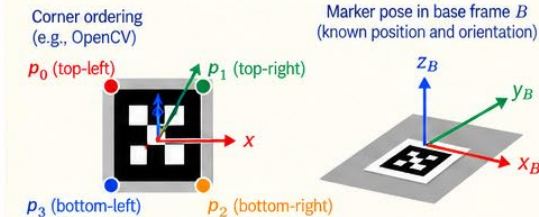
Determine the camera matrix K and distortion coefficients d for the image resolution used by `vision.py`.

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \quad d = [k_1, k_2, p_1, p_2, k_3]$$

- Obtain K and d using a standard camera calibration (e.g., OpenCV calibration with a chessboard).
- Account for distortion when processing detected image coordinates (use `cv2.undistortPoints` or equivalent).

3. Marker Geometry

For each detected ArUco marker, compute the 3D coordinates of its four corners in the robot base frame B from its known size, position, and orientation.



- Use the known side length and pose (position and orientation).
- Specify the corner ordering and associate each detected marker with its geometry using its ArUco ID.

5. Transform Detected Object Positions

For a detected image point (u, v) , obtain its camera ray r_C from K :

$$\tilde{p}_C = K^{-1} [u, v, 1]^T, \quad r_C = \frac{\tilde{p}_C}{\|\tilde{p}_C\|}$$

Transform the ray to the robot base frame:

$$P_B(\lambda) = t_{BC} + \lambda R_{BC} r_C$$

Intersect the ray with the plane $z_B = z_{object}$ (top surface of the cube):

$$\lambda = \frac{z_{object} - t_{BC,z}}{(R_{BC} r_C)_z}, \quad P_B = t_{BC} + \lambda R_{BC} r_C$$

- Specify the relevant surface height z_{object} in the robot base frame.
- Reject intersections outside the robot workspace or behind the camera.
- Use this calibrated transformation in `send_cube_coords()`.

6. Validate the Transformation

Evaluate the calibration quality and accuracy.

- Reproject the known marker corners and compute the mean and maximum reprojection errors (in pixels).
- Place a cube at several independently measured workspace positions (including near the edges) and compare the returned robot coordinates with the measured coordinates (mean and maximum position errors in millimeters).
- Repeat the calibration with several images and assess its consistency.

7. Deliverables

- Updated `vision.py` using the calibrated transformation.
- Calibration parameters (K , d , R_{BC} , t_{BC}) with coordinate frame and unit conventions.
- Annotated image showing detected and reprojected marker corners.
- Table of validation errors (reprojection error in pixels and position error in millimeters).

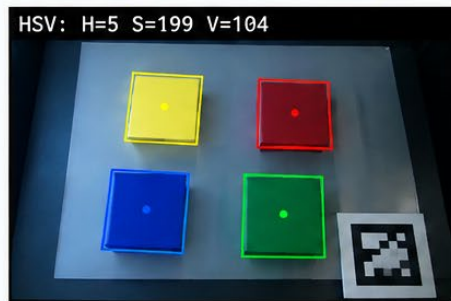
Extension Robust HSV Segmentation

Task: Robust HSV Segmentation for Cube Detection

Improve the HSV masking so that cube detection is reliable under different lighting conditions

1. Existing Implementation

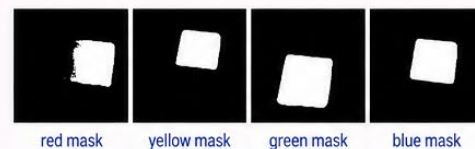
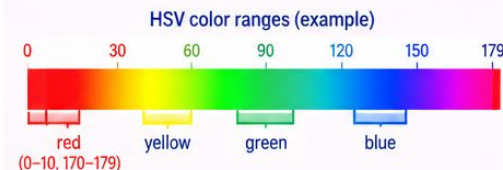
- Camera image (BGR) → convert to HSV.
- Fixed HSV thresholds for red, yellow, green, blue (red uses two hue intervals).
- Binary masks → find contours → `cv2.minAreaRect()`.
- Select approximately square regions (side length 50–150 px, aspect ratio close to 1).
- Store center and orientation in `self.detected_cubes`.
- Mouse overlay shows HSV values at current pixel.



Detected cubes with bounding rectangles and center points

2. HSV Color Segmentation

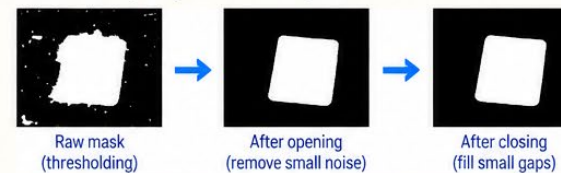
- Convert image from BGR to HSV.
- Threshold each color using `cv2.inRange()`.
- Red uses two hue ranges (hue wraps around 0/179).
- Clean the binary mask with morphological operations (opening/closing) before contour detection.



Binary masks for each color (after thresholding and cleaning)

3. Mask Cleaning with Morphology

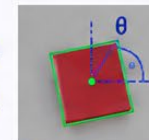
- Remove small isolated regions and fill small gaps.
- Typical operations: opening (erode → dilate) to remove noise, closing (dilate → erode) to fill holes.
- Choose and justify the structuring element and its size.



Effect of morphological operations (example for red mask)

4. Contour Filtering and Cube Detection

- Find contours with `cv2.findContours()`.
- Approximate the contour with `cv2.minAreaRect()`.
- Filter candidates:
 - Area within a valid range (e.g., 50–150 px side length).
 - Aspect ratio close to 1 (approximately square).
 - Optional: check that the candidate lies within the workspace.



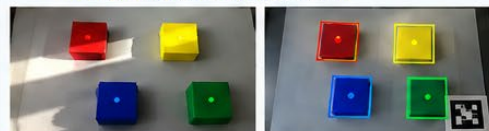
Detected contour with `minAreaRect()`

5. Prevent Stale Detections

- Create a new detection dictionary at the start of each image callback and replace `self.detected_cubes` after processing all colors.
- Only report a color if a valid cube of that color is detected in the current frame.
- Ensure `send_cube_coords()` returns the existing “not detected” response when a cube is removed. (Do not keep detections from earlier frames.)

6. Handling Challenging Conditions

- Test with different lighting conditions:
 - normal lighting
 - shadows
 - reflections
 - changed brightness / white balance



Challenging lighting (e.g., shadows, reflections)

Robust detection result

7. Evaluation and Deliverables

- Test the detector with cubes in several positions and orientations.
- Include cases with shadows or reflections and a frame where a previously detected cube has been removed.
- Show original and improved masks for representative frames.
- Report false detections or missed cubes.
- Explain your threshold choices and filtering criteria.
- The node must continue to provide cube coordinates through the existing `/cube_coordinates` service.

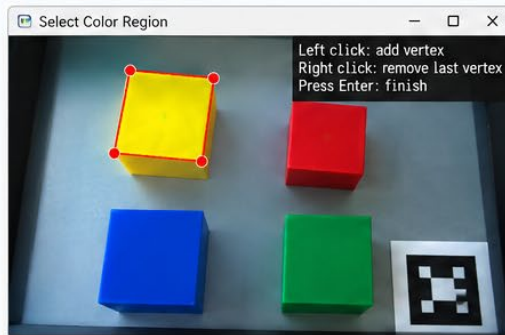
Extension Interactive HSV Segmentation Calibration

Task: Interactive Calibration of HSV Thresholds

Estimate suitable HSV thresholds from a selected object in a camera image

1. Select a Colored Object

- Load an image and display it in an OpenCV window.
- Let the user mark the boundary of a colored object by clicking its vertices.
- Right-click removes the last vertex; press Enter to complete the polygon.



2. Create Mask and Extract HSV Pixels

- Create a binary mask of the polygon using `cv2.fillPoly()`.
- Convert the image from BGR to HSV and extract only the HSV pixels inside the selected region.
- Exclude a narrow band along the polygon boundary to reduce background influence.



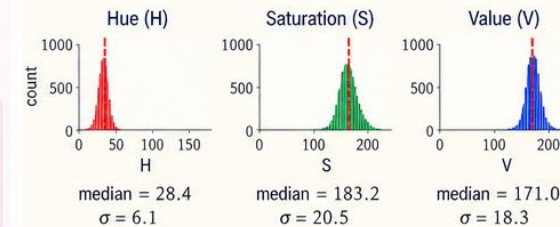
```
mask = np.zeros(image.shape[:2], dtype=np.uint8)
cv2.fillPoly(mask, [pts], 255)
mask_inner = cv2.erode(mask, np.ones((k,k), np.uint8)) # exclude border
hsv = cv2.cvtColor(image, cv2.COLOR_BGR2HSV)
pixels = hsv[mask_inner > 0]
```

3. Analyze HSV Distribution

- Compute the median and standard deviation for hue, saturation, and value.
- Determine preliminary thresholds using

$$x_{\min} = \max(0, \text{median}(x) - 2\sigma_x),$$
$$x_{\max} = \min(255, \text{median}(x) + 2\sigma_x), \quad x \in \{S, V\}.$$

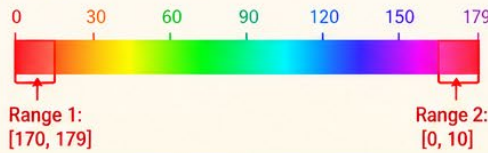
For hue H (range 0–179), handle wrap-around:
values near 179 and 0 may describe the same color.



4. Determine Final Thresholds

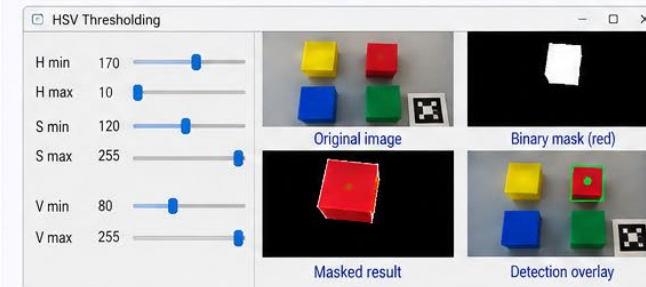
- For each channel, use the computed min/max values.
- For hue, check if the interval crosses 0 or 179. If so, use two ranges and combine the masks with `cv2.bitwise_or()`.
- Optionally, adjust the thresholds manually and visualize the result with trackbars.

Example: hue wrap-around for red



5. Apply Thresholds and Visualize Result

- Apply the thresholds to the entire image using `cv2.inRange()`.
- Display the original image, the mask, and the masked result.
- Provide OpenCV trackbars to adjust the thresholds interactively.



6. Save and Use Thresholds

- Print the resulting threshold values.
- Store them in a format that can be copied into the colors dictionary of `vision.py`.

Example output for red:

```
Red thresholds (HSV):
H: [170, 179] and [0, 10]
S: [120, 255]
V: [80, 255]

colors = {
    'red': {
        'h': [(170, 179), (0, 10)],
        's': (120, 255),
        'v': (80, 255)
    },
    ...
}
```

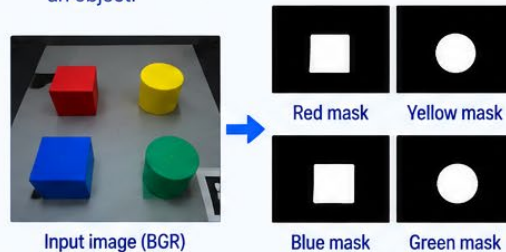
Extension Object Shape Detection

Task: Detecting Object Color and Shape

Extend the vision module to detect both the color and the shape of objects (cubes and cylinders)

1. Segment Objects by Color

- Use the HSV ranges to create a mask for each supported color (red, yellow, green, blue).
- Apply mask cleanup (e.g., opening/closing) before contour extraction.
- Discard regions that are too small to represent an object.

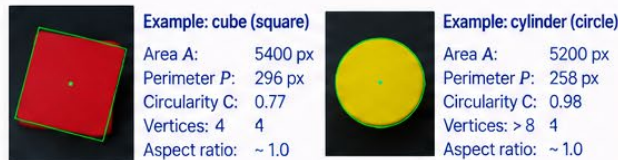


2. Extract Shape Features

- For each contour, compute area A , perimeter P , center, minimum-area bounding rectangle, and circularity.
- Approximate the contour using `cv2.approxPolyDP()` and inspect its number of vertices.

Circularity:
$$C = \frac{4\pi A}{P^2}$$

 A ... contour area
 P ... contour perimeter
 C ... circularity (1 for a perfect circle)



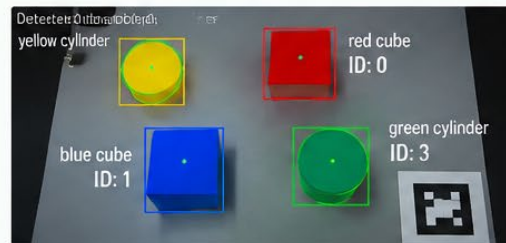
3. Classify Color and Shape

- Assign a color from the HSV mask.
- Classify the shape as cube or cylinder using contour features (e.g., circularity, number of vertices, aspect ratio).
- Define and justify thresholds using real images.
- Mark uncertain regions as unknown.



4. Store and Display Detections

- Replace the data structure so that multiple objects (and the same color) can be stored.
- Store for each object: color, shape, image center, and orientation (if meaningful).
- Draw each contour, label (color + shape), and center in the debug window.
- Remove detections when objects disappear (do not keep stale detections from earlier frames).



5. Extended Query Interface

- Extend the service so that a client can request an object by both color and shape (e.g., color = red, shape = cylinder).
- Return a unique object ID and its pose/coordinates.
- Specify the behavior when no object matches or when multiple objects match (e.g., return all matches or the closest one).

Current service (simplified)

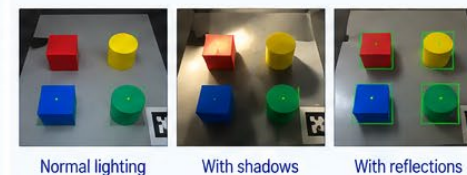
```
GetCubeCoords.srv
string color
---
float32 x
float32 y
float32 z
float32 yaw
# not detected: [-1, -1, -1]
```

Extended service (example)

```
GetObject.srv
string color # e.g., "red"
string shape # e.g., "cube" | "cylinder"
---
bool found
int32 id # unique object ID
geometry_msgs/Pose pose
# optionally: list of all matches
```

6. Evaluation and Examples

- Test with cubes and cylinders in several positions and orientations.
- Use different lighting conditions, shadows, and reflections.
- Report correct detections, missed objects, and false detections.
- Show example debug images and justify your classification thresholds.



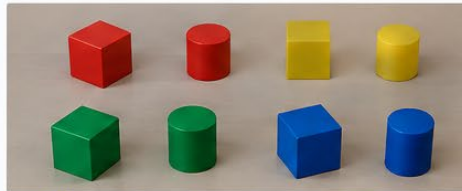
Assignment Object Interface

Task: Extension of the Object Interface

Select objects by both color and shape and represent them correctly in the planning scene

1. Current Limitation

- Objects are identified only by color.
- `get_cube_coords(color)` returns one cube and the collision-object ID is the color string.
- All objects are represented in the planning scene as a box of fixed size $0.04 \times 0.04 \times 0.04$ m.
- Cannot distinguish a red cube from a red cylinder or multiple objects of the same shape.



Different colors and shapes (cubes and cylinders)

2. Extend the Vision Service

- Add a shape field to the request so that objects can be selected by color and shape.
- Replace the special coordinate array with an explicit Boolean found field.
- Return a unique object ID together with the pose. If several objects match, specify how one is selected or return all available matches.

Current service (simplified)

```
# GetCubeCoords.srv (current)
string color
---
float32 x
float32 y
float32 z
float32 yaw
# not detected: [-1, -1, -1, -1]
```



Extended service (example)

```
# GetObject.srv (proposed)
string color # e.g., "red"
string shape # e.g., "cube" | "cylinder"
---
bool found
int32 id # unique object ID
geometry_msgs/Pose pose
# optional: list of all matches
# Object[] candidates
```

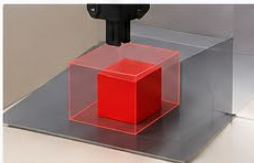
3. Update the Service Client

- Replace `get_cube_coords(color)` with `get_object(color, shape)`.
- Set both request fields and check `response.found`.
- Pass the returned ID and pose to the manipulation functions (pick, drop, place).

```
# Example service client call (Python)
req = GetObject.Request()
req.color = 'red'
req.shape = 'cylinder'
resp = self.object_client.call(req)
if resp.found:
    obj_id = resp.id
    pose = resp.pose
    print(f"Found object {obj_id}: {pose}")
else:
    print("No matching object found.")
```

4. Represent Detected Shape in MoveIt

- Use the detected shape to add the correct collision object to the planning scene:
 - Cube → `SolidPrimitive.BOX`
 - Cylinder → `SolidPrimitive.CYLINDER`
- Use calibrated object dimensions.
- Identify collision objects by unique IDs (not by color).
- Update add, attach, detach, and remove operations accordingly.



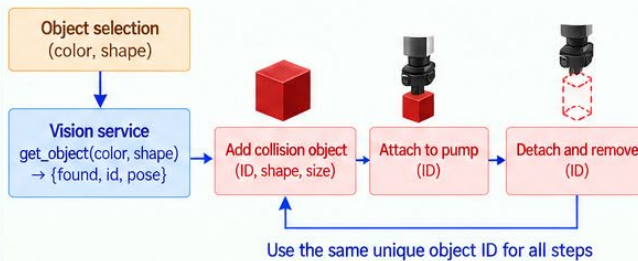
Cube as collision object
(SolidPrimitive.BOX)



Cylinder as collision object
(SolidPrimitive.CYLINDER)

5. Pass Object Identity Through Manipulation

- Use the selected object's ID and pose in `pick()` and `drop()`.
- Use the same ID when adding the object to the planning scene, attaching it to the pump, detaching it, and removing it.
- Review `place()` separately: its fixed height increment of 0.04 m assumes a stack of cubes (adapt if cylinders are used).



6. Extend the Terminal Menu

- In `choose_pick()`, ask the user to select a color and a shape.
- If multiple objects match, display their IDs and positions and let the user select one.
- Keep the destination selection in `choose_drop()`.
- Simplify the menu control flow by using loops instead of repeated function calls.

```
=== Pick and Place Control ===
1 - Stacking mode
2 - Sorting mode
3 - Pick object (select color and shape)
4 - Move to home
5 - Open gripper (pump off)
6 - Close gripper (pump on)
7 - Quit

Select a color: [red, yellow, green, blue]: red
Select a shape: [cube, cylinder]: cylinder
Found 2 matching objects:
ID 3: position (0.12, -0.05, 0.01)
ID 5: position (0.13, 0.04, 0.01)
Select object ID: 3
```